



INSTITUTO POLITÉCNICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA Y CIENCIAS SOCIALES Y ADMINISTRATIVAS

MATERIA: ABSTRACCIÓN Y USO DE DATOS

Reporte de prácticas del 3er Parcial

GRUPO: 3CM30

PROFESOR: Yventz Entzana Garduño

ALUMNOS: Cruz Ortega Omar Josue

No. Lista

5

Mondragon Ortiz Itzel Andrea

27

Contenido

Enlace: Repositorio de GitHub.....	4
Códigos del Primer Parcial.....	4
Código 1: “Hola Mundo”.....	4
Código 2: “Suma de 2 números con parámetros”.....	5
Código 3: “Calculadora Básica”.....	5
Código 4: “2 parámetros, 0 parámetros o 3 parámetros”.....	6
Código 5: “herencia- calculadora”.....	8
Código 6: “Herencia y sobreescritura”.....	9
Código 7: “Creación de datos”.....	9
Código 7bis: “Identificar tamaños”.....	10
Código 8: “Promedio de 5 números”.....	11
Código 9: “Promedio de 5 números con un arreglo”.....	12
Código 10: “Promedio de 5 números con puntero al arreglo”.....	13
Código 11: “Calculo de matrices”.....	14
Código 12: “Nuevo tipo de dato como arreglo”.....	15
Código 13: “Nuevo tipo de dato con puntero al arreglo”.....	15
Código 14: “Recursividad”.....	16
Código 15: “Triangulo Sierpinski”.....	17
Códigos del segundo Parcial.....	18
ADT.....	18
Arreglo - Pila, Cola y Lista (P3 a la P5).....	18
P3 - Análisis Lógico de la Pila (Stack).....	18
P4- Análisis Lógico de la Cola (Queue).....	19
P5- Análisis Lógico de la Lista.....	21
Puntero - Pila, Cola y Lista (P6 a la P7Bis).....	22
P6- Lógica y Sintaxis de la Pila.....	22
P7- Lógica y Sintaxis de la Cola.....	23
P7BIS - Lógica y Sintaxis de la Lista.....	25
Librerías - Pila, Cola y Lista (P8 a la P10).....	26
P8 - Lógica y Sintaxis de la Pila.....	26

P9 - Lógica y Sintaxis de la Cola	27
P10 - Lógica y Sintaxis de la Lista.....	29
ARCHIVOS.....	30
Escribir y leer CSV datos básicos, arreglos, creados arreglos y los ADT	33
Escribir y leer XML datos básicos, arreglos, creados arreglos y los ADT.....	35
Escribir y leer txt datos básicos, arreglos, creados arreglos y los ADT	36
Escribir y leer Json datos básicos, arreglos, creados arreglos y los ADT	38
GRAFO.....	40
Grafo con Algoritmo de Dijkstra y Exportación SVG.....	40
DÍGRAFO	43
TDA Digrafo (Grafo Dirigido) y Serialización JSON	43
ÁRBOL.....	45
TDA Árbol y Jerarquías Visuales en SVG.....	45

Enlace: Repositorio de GitHub

Enlace al repositorio : [OmarC777/Practicas-3er-ORD: Contiene las practicas solicitadas para el tercer ordinario](#)

Códigos del Primer Parcial

Código 1: “Hola Mundo”

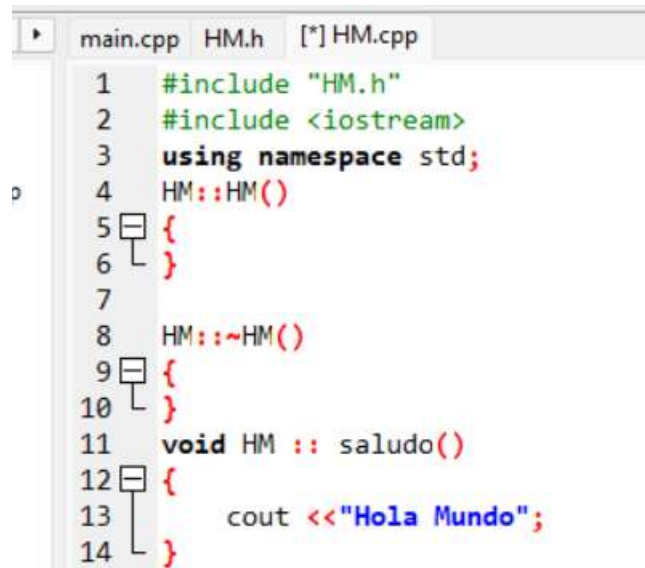
Objetivo: Desarrollar una estructura que imprima un “Hola mundo”

¿De qué trata el código?

Para este primer programa fue sencillo, ya que como dice en el objetivo este va a imprimir un saludo, en este caso “Hola mundo”

Para esto se crean tres archivos: el main; donde se llama al método, en este caso el `obsal.saludo()`; el `HM.h`, para crear la clase y el `.cpp`, donde se desarrolla el Código.

No tuvimos ninguna complicación al realizar esta práctica.



```
main.cpp  HM.h  [*] HM.cpp
1  #include "HM.h"
2  #include <iostream>
3  using namespace std;
4  HM::HM()
5  {
6  }
7
8  HM::~~HM()
9  {
10 }
11 void HM :: saludo()
12 {
13     cout <<"Hola Mundo";
14 }
```

Código 2: “Suma de 2 números con parámetros”

Objetivo: Desarrollar una estructura de programación orientada a objetos en C++ que permita realizar operaciones aritméticas básicas.

¿De qué trata el código?

En este código se realizó la suma de dos números, usando una parte fundamental de la programación orientada a objetos, donde en el usuario ya no tiene que teclear nada, ya que en el mismo código se implementa el valor de los números, por lo que también fue una práctica sencilla

Dificultades

Un error muy pequeño que tuvimos fue la implementación de las librerías, ya que en la función principal (main), no habíamos declarado la librería del archivo .h (donde se crean las clases)

¿Cómo funciona el Código?

Declaración: Crea una clase llamada Operacion que funciona como una "plantilla" para realizar la suma

Se llama a la función cuando el programa principal invoca a Suma(int a, int b), el procesador toma los valores de entrada que en este caso fueron 5 y 3, donde ejecuta la operación aritmética y expulsa el resultado

en este caso fue en el **int res = obS.Sum(num1, num2);**: Aquí se le da un orden al objeto como diciéndole “usa la función que tienes, los sumas y me arrojas el valor” y el valor se guarda en la variable **res**

Mientras, el constructor y destructor se aseguran de que el objeto se cree y se destruya correctamente dentro del sistema.

Código 3: “Calculadora Básica”

Objetivo: Desarrollar una estructura de programación orientada a objetos en C++ que permita realizar operaciones aritméticas básicas

¿De qué trata el código?

Para esta práctica se desarrolló una calculadora muy básica con conceptos básicos de POO, donde al usuario le preguntaran que operación aritmética desea realizar y con base a eso le pedirán 2 números, para que el programa haga la operación que se le solicito (suma, resta, multiplicación o división).

Dificultades

Como en la práctica anterior, se tuvo un error muy pequeño que fue la implementación de las librerías, ya que en la función principal (main), no habíamos declarado la librería del archivo .h (donde se crean las clases) y, por ende, el programa no se ejecutaba de manera correcta, porque no se mandaba a llamar nada.

¿Cómo funciona el Código?

La función principal: solo se creó el objeto para poder mandar a llamar a la función y aparte es como el recepcionista del programa

.cpp: Aquí es cuando empieza el desarrollo del código:

En lugar de tener todo el código amontonado en una sola función, se decide dividir cada tarea en diferentes bloques (métodos)

- **Suma, Resta, multiplicación y división:** Cada uno se realizó independiente, por lo que tienen sus propias variables locales (s1, s2, resS, etc.) y así capturar los datos que el usuario ingrese tecleando y poder mostrar el resultado.
- **Uso de double:** Empleamos este tipo de dato para permitir que el usuario al ingresar sus valores funcione tanto con números enteros como con decimales.

Y para dar a elegir qué operación se quiere realizar, se creó un menú, para esto usamos un **switch** para decidir que método ejecutar. Por ejemplo, si el usuario presiona "3", el programa salta directamente a la función de multiplicación.

Código 4: “2 parámetros, 0 parámetros o 3 parámetros”

Objetivo: Desarrollar una estructura de programación orientada a objetos en C++ que permita realizar operaciones aritméticas utilizando parámetros y sobrecarga

¿De qué trata el código?

Para esta práctica se desarrolló un programa, donde al principio se utilizan 2 y 3 parámetros, en este caso ya nos da el resultado con la operación que se está realizando, para luego, pedirle al usuario 2 números, para que el programa haga la operación.

El código tiene tres métodos que se llaman exactamente igual: Sumar. Sin embargo, la computadora no se confunde porque cada uno tiene una "firma" diferente (tipo de datos de entrada).

Dificultades

Se tuvo como dificultad el que, al momento de implementar los métodos, como se está usando parámetros, nos percatamos de que el error en la definición del método `double Calculadora::Sumar(double n1, double n2)`. El problema radicaba en la falta de correspondencia entre los parámetros enviados y los recibidos, por lo que solo se lo agregamos (en el segundo y tercer parámetro)

¿Cómo funciona el Código?

Como ya se había dicho, su funcionamiento principal se basa en la sobrecarga de funciones, permitiendo que un mismo nombre de método realice tareas distintas según los datos recibidos.

- La primera versión de Sumar interactúa con el usuario pidiendo valores por consola, mientras que los otros dos métodos, estos se procesan directamente con los valores que ya se les asignó dentro del código (dos o tres parámetros).
- El sistema utiliza el tipo de dato `double` para asegurar precisión en cálculos con decimales y valores de retorno para devolver los resultados

Los constructores y destructores están presentes para administrar el ciclo de vida de los objetos creados. Finalmente, esta organización modular facilita la lectura, el mantenimiento y la futura expansión de funciones matemáticas

```
double Calculadora::Sumar(double n1, double n2) //
{
    cout << "Suma con dos parametros" << endl;
    cout << n1 << " + " << n2 << " = ";
    double res = n1 + n2;
    return res;
```

Código 5: “herencia- calculadora”

Objetivo: Desarrollar una estructura de programación orientada a objetos en C++ que permita realizar operaciones aritméticas utilizando la herencia.

¿De qué trata el código?

Solo para esta práctica se tuvo que agregar la herencia que se pedía en las instrucciones, básicamente hace lo mismo, solo que aquí se utiliza la herencia para que la clase Científica reutilice todo el código de Calculadora sin tener que volver a escribir las funciones de suma, resta, etc., añadiendo la capacidad de calcular potencias.

Dificultades

Solo se tuvo el error de la sintaxis al momento de crear la herencia.

¿Cómo funciona el Código?

El código lo que hace es que permite que una misma función actúe de tres formas distintas según los datos que reciba: puede pedir números al usuario por consola si no recibe parámetros, o sumar directamente dos o tres valores si se le pasan como argumentos. Cuando el programa se ejecuta, el compilador identifica cuál de estas versiones usar basándose en la "firma" de la función (el número de datos enviados). Por otro lado, el método Potencia de la clase Científica utiliza la función predefinida pow() para elevar una base a un exponente, demostrando cómo se pueden integrar herramientas matemáticas estándar dentro de una estructura de clases personalizada.

```
#ifndef CIENTIFICA_H
#define CIENTIFICA_H
class Calculadora
{
public:
    Sumar();
    double Sumar(double n1, double n2);
    double Sumar(double num1, double num2, double num3); //
    Calculadora();
    ~Calculadora();
protected:
};

class Cientifica : public Calculadora
{
public:
    double Potencia(double base, double exponente);
    double MultiplicacionSucesiva(double base, double exp); //
    double DivisionSucesiva(double dividendo, double divisor);
    Cientifica();
    ~Cientifica();
protected:
};
```


Código 6: “Herencia y sobreescritura”

¿De qué trata este código?

Para esta práctica de tercer semestre, desarrollé un programa en C++ que simula dos calculadoras: una básica y una científica. El objetivo principal era aprender a organizar el código y utilizar un concepto llamado "herencia". Esto básicamente sirve para que una parte nueva del programa pueda usar y mejorar el código que ya escribimos antes, sin tener que volver a teclearlo desde cero.

Dificultades.

Tuvimos varios errores a la hora de poner la herencia. La idea era que la calculadora científica (que sería la "hija") tomara prestadas las herramientas de la calculadora básica (la "padre"). Sin embargo, el código nos marcaba error porque se confundía. Como tenía varias formas de hacer una "suma", al conectar las dos calculadoras la computadora no sabía bien cuál suma usar o bloqueaba el acceso a las funciones originales. Me costó un poco de trabajo ajustar los permisos y organizar el código para que las dos calculadoras se comunicaran correctamente sin chocar.

¿Qué hace el programa?

- **Diferentes formas de sumar:** El programa sabe qué hacer dependiendo de lo que le pidas. Puede sumar pidiéndote los números directo en la pantalla, o puede hacer la suma si tú le das dos o tres números por detrás del código.
- **Matemáticas con ciclos:** Para la calculadora científica, en lugar de usar los atajos que ya trae la computadora para multiplicar o dividir, usamos lógica básica. Por ejemplo, la multiplicación la hace sumando el mismo número varias veces, y la división la calcula haciendo restas repetidas.

Código 7: “Creación de datos”

Objetivo: Desarrollar una estructura de programación orientada a objetos en C++ que permita imprimir datos personales usando el “struct”

¿De qué trata el código?

Esta práctica estuvo sencilla, ya que ahora en vez de usar clases, implementamos un “struct” que básicamente es lo mismo (comparten la misma sintaxis), para esto, solo se le pedirán al usuario datos básicos.

¿Cómo funciona el Código?

Definición de Moldes: Se crean dos estructuras (Auto y Persona). Una estructura es como un plano que define que datos (variables) y que acciones (funciones) tendrá un objeto.

Instanciación: En la función main(), se crea una variable llamada p1 de tipo Persona. Esto reserva espacio en memoria para guardar el nombre, apellidos, edad y genero de esa persona específica.

Interacción (Entrada): Se llama al método p1.capturar(). El programa se detiene y espera a que el usuario escriba los datos que le solicita el programa.

Salida de Datos: Finalmente, se ejecuta p1.saludar(), que toma los datos guardados en la memoria de p1 y los imprime.

Código 7bis: “Identificar tamaños”

Descripción del código

El objetivo principal del programa es ver cómo se almacenan las clases y estructuras en la memoria utilizando C++. Usando el operador sizeof, el código calcula el espacio (en bytes) que ocupan diferentes elementos: variables simples, objetos de la clase Alumno y la estructura AlumnoS.

Un ejemplo de como se uso el operador sizeof es: cout <<

"tamaño de un entero --> " <<sizeof(AlumnoS) <<endl

<<endl <<endl;

Esto determina cuantos bytes tiene la estructura de “AlumnoS”

Implementación y adiciones

Se dividió el código en tres partes:

1. Declaración de Clase (Alumno.h): Creamos una clase con atributos privados y públicos, incluyendo arreglos de caracteres para nombre y boleta, y un dato tipo double para el promedio.

2. Creación de Métodos (Alumno.cpp): Declaramos los constructores, destructores y funciones de impresión (msg) para asegurar que el objeto funcionara.
3. Cálculo de Memoria (main.cpp): Agregamos lógica para imprimir el tamaño de:

Argumentos del sistema (argc).

Instancias de la clase Alumno.

La estructura AlumnoS.

Errores

Mientras desarrollábamos el código tuvimos varias complicaciones, entre ellas las siguientes:

1. Error en la inclusión de la clase: Al principio, el compilador no encontraba la clase Alumno. Lo solucionamos poniendo `#include "Alumno.h"` tanto en el .cpp de la clase como en el main.
2. Confusión con el Tamaño Real: Notamos que el tamaño de la estructura no siempre era la suma exacta de sus partes. Investigamos por nuestra cuenta y aprendimos que esto se debe al Memory Padding (relleno), donde el compilador alinea los datos para mejorar la velocidad de procesamiento.
3. Uso de namespace: Olvidamos el `using namespace std;` en algunos archivos, lo que causaba errores en los `cout`. Se corrigió agregándolos en los archivos donde faltaban (main y en el Alumno.cpp).

Código 8: “Promedio de 5 números”

¿De qué trata este código?

Para esta práctica desarrollamos un programa en C++ que le pide al usuario ingresar 5 números. A partir de esos datos, la computadora hace los cálculos necesarios para dar cuatro resultados finales: la suma de todos los números, su promedio, cuál de ellos fue el más grande (máximo) y cuál el más pequeño (mínimo).

¿Cómo funciona el código?

Para no tener que escribir el código de "pedir número" cinco veces , utilizamos un ciclo for.

- **Repetición automática:** El ciclo da 5 vueltas, pidiendo un dato nuevo en cada una.
- **Cálculo:** Conforme vas metiendo los números, el programa los va sumando y guardando en una especie de "alcancía" para después calcular el promedio. No espera hasta el final para hacer las sumas.
- **Máximo y Mínimo:** Al principio tomamos el primer número como el mas pequeño y el mas grande al mismo tiempo. Después, cuando llegan los números 2, 3, 4 y 5, simplemente se compara con los que ya estaban guardados y se reemplazan si se encuentra uno que sea mayor.

```
void Promedio::Operaciones()
{
    double suma = 0, numeros, maximo, minimo, prom;
    for (int i = 1; i <= 5; i++)
    {
        cout << "Ingresa valor No. " << i << " para hacer el calculo" << endl;
        cin >> numeros;

        suma += numeros;

        prom = suma / i;

        if( i == 1)
        {
            maximo = numeros;
            minimo = numeros;
        }
        else
        {
            if (numeros > maximo)
            {
                maximo = numeros;
            }
            if (numeros < minimo)
            {
                minimo = numeros;
            }
        }
    }
}
```

Código 9: “Promedio de 5 números con un arreglo”

¿De qué trata este código?

Esta práctica es una mejora del programa anterior donde calculamos la suma, el promedio, el máximo y el mínimo de 5 números. La gran diferencia es que utilizamos un arreglo. En lugar de calcular todo al momento y descartar los números introducidos, el programa primero guarda los 5 datos en un arreglo y procede a hacer los cálculos.

Dificultades

Tuvimos algunos errores a la hora de implementar el arreglo junto con el ciclo for. El principal problema fue acostumbrarnos a cómo cuenta la computadora. En los arreglos las casillas se enumeran del 0 al 4. Al principio el ciclo intentaba guardar o leer datos en la posición "5" del arreglo, lo que hacía que el programa compilara mal porque esa casilla no existía. Pero se pudo ajustar los límites de la variable *i* para que coincidieran con las posiciones reales del arreglo.

¿Cómo funciona el código?

- **Recolección de datos:** El primer ciclo se dedica exclusivamente a pedirle al usuario los 5 números y guardarlos en el arreglo "numeros[5]".
- **Cálculos:** Una vez que el arreglo está lleno, toma el primer número que ingresó el usuario y lo declara como el "máximo" y el "mínimo" al mismo tiempo. Después, el segundo ciclo recorre toda la lista paso a paso; va sumando todos los valores y va comparando cada casilla para ver si encuentra un número más grande o más chico. Al terminar el recorrido, simplemente calcula el promedio dividiendo entre 5.

Código 10: "Promedio de 5 números con puntero al arreglo"

¿De qué trata este código?

Esta es la tercera versión de nuestro programa para el promedio de 5 números. Al igual que en la versión anterior, el código guarda los números en un arreglo, los suma, saca el promedio y descubre cuál es el mayor y el menor. Lo nuevo es que en esta práctica agregamos un puntero.

¿Qué hay de nuevo en esta versión?

- **Creación del puntero:** En el código, creamos un puntero llamado "pun" y apunta directamente a el arreglo de "numeros[5]". El simple hecho de escribir el nombre de un arreglo ya nos da la dirección de la primera casilla, así que el puntero se quedó "apuntando" al inicio de nuestro arreglo.
- **Valor del puntero:** El programa hace todos los cálculos matemáticos tal como lo hacía antes, pero al final añadimos una instrucción extra. Le pedimos a la computadora que imprimiera en pantalla el valor de nuestro puntero.

```

void PromedioP::OperacionesP()
{
    double suma = 0, numeros[5], maximo = 0, minimo = 0, prom = 0, *pun = numeros;
    for (int i = 0; i <= 4; i++)
    {
        cout << "Ingresa valor No. " << i << " para hacer el calculo" << endl;
        cin >> numeros[i];
    }

    suma = 0;
    maximo = numeros[0];
    minimo = numeros[0];
    for (int i = 0; i <=4; i++)
    {
        suma = suma + numeros[i];
        if (numeros[i] > maximo)
        {
            maximo = numeros[i];
        }
        if (numeros[i] < minimo)
        {
            minimo = numeros[i];
        }
    }
    prom = suma / 5;
}

```

Código 11: "Calculo de matrices"

¿De qué trata este código?

Construimos un programa en C++ capaz de trabajar con "arreglos bidimensionales", que básicamente son tablas o cuadrículas de números (matrices).

Dificultades

Tuvimos varias dificultades a la hora de hacer y estructurar las matrices, fue bastante confuso. La mayor dificultad fue coordinar los ciclos for para no salirnos de los límites del arreglo. Especialmente en la multiplicación, hacer que el código multiplicara las filas de la primera tabla por las columnas de la segunda hizo que nos confundiéramos varias veces antes de que el resultado saliera bien.

¿Qué hace el programa?

- **Multiplicar una tabla por un número:** El código toma todos los números dentro de la matriz y los multiplica por un solo número que nosotros elegimos (en este caso, un 5). Es decir, hace el cálculo cuadrado por cuadrado.
- **Multiplicar dos tablas entre sí:** Esta fue la parte más difícil. El código es capaz de tomar dos matrices de 2x2 y multiplicarlas utilizando la regla matemática clásica. Para hacer esto, tuvimos que meter un ciclo for dentro de otro, y luego otro más adentro del anterior (tres en total) para ir sumando y guardando los resultados en una tabla nueva (arreglo).

Código 12: “Nuevo tipo de dato como arreglo”

¿De qué trata este código?

El objetivo fue crear un sistema de registro que almacena la información de 5 personas y los datos del automóvil de cada una de ellas. Para hacer esto usamos "estructuras" (struct), que son como plantillas o formularios en blanco que nosotros mismos diseñamos.

¿Cómo funciona el código?

- **Captura de datos:** El primer ciclo da 5 vueltas pidiéndote los datos. Dentro del mismo ciclo damos la instrucción y la dirección de donde guardar los registros. Por ejemplo, al escribir `lista[i].vehiculo.precio`, estamos dando una ruta exacta.
- **Impresión de resultados:** Una vez que terminamos de llenar todos los datos de las 5 personas, arranca un segundo ciclo. Este ciclo simplemente recorre la lista y ejecuta una función llamada `mostrarInformacion()`, esto imprime en pantalla los datos del dueño y su auto.

```
struct Persona
{
    string nombre;
    string ap;
    string am;
    string genero;
    int edad;
    Auto vehiculo;

    void mostrarInformacion() {
        cout << "Nombre: " << nombre << endl << "Apellido Paterno: " << ap << endl << "Apellido Materno: " << am << endl;
        cout << "Genero: " << genero << endl << "Edad: " << edad << endl;
        cout << "Precio del auto: $" << vehiculo.precio << endl << "Modelo: " << vehiculo.anio << endl;
        cout << "_____" << endl;
    }
}
```

Código 13: “Nuevo tipo de dato con puntero al arreglo”

¿De qué trata este código?

Esta práctica es una nueva versión de nuestro sistema de registros anterior. Seguimos utilizando las "estructuras" (struct) para guardar la información de 5 personas y los datos de sus automóviles (precio y año).

¿Qué hay de nuevo?

Justo después de crear nuestra lista en blanco para 5 personas (`Persona lista[5];`), agregamos nuestro apuntador: `Persona *pun = lista;`

En otras palabras, lo que hicimos aquí fue crear una flecha virtual (el apuntador llamado pun) y apunta directamente a la primera persona de nuestra lista

```
int main()
{
    Persona lista[5];
    Persona *pun = lista;
    for (int i = 0; i < 5; i++)
    {
        cout << "-----Datos Persona-----" << endl;
        cout << "Nombre: "; cin >> lista[i].nombre;
        cout << "Ap. Paterno: "; cin >> lista[i].ap;
        cout << "Ap. Materno: "; cin >> lista[i].am;
        cout << "Edad: "; cin >> lista[i].edad;
        cout << "Genero: "; cin >> lista[i].genero;
        cout << ">>>>>>>Datos de su auto<<<<<<<" << endl;
        cout << "Precio: "; cin >> lista[i].vehiculo.precio;
        cout << "Anio: "; cin >> lista[i].vehiculo.anio;
    }

    cout << "-----Registros-----" << endl;
    for (int i = 0; i < 5; i++)
    {
        lista[i].mostrarInformacion();
    }

    return 0;
}
```

Código 14: “Recursividad”

Objetivo: Desarrollar una estructura de programación orientada a objetos en C++ que permita extender las funciones de una calculadora básica para realizar operaciones complejas (potencias, factoriales, fibonacci).

¿De qué trata el código?

Esta práctica trata de al momento de ingresar datos, nos arrojen más operaciones, en este caso potencias, factoriales, etc.. obviamente con sus respectivos resultados.

Pero para esta práctica, vamos a estar partiendo de la practica 6, por lo que usamos dos clases la “Calculadora” y “Científica”

La clase Calculadora: Utiliza sobrecarga de funciones, lo que permite tener varios métodos llamados Sumar que se comportan distinto según cuántos números tenga

Y la clase Científica: Aquí se implementan algoritmos matemáticos clásicos como la serie de Fibonacci y el Factorial, además de recrear operaciones básicas (como

la multiplicación y la división) de forma manual mediante sumas y restas sucesivas.

Dificultades

Se complico al momento de empezar a crear con lo de Fibonacci, lo factorial y más que nada la sintaxis, el problema fue como desarrollar esas operaciones dentro del código y en la recursividad.

¿Qué hace el Código?

En la clase base, el programa decide qué función ejecutar basándose en los parámetros:

- Si llamas a Sumar(), te pide los datos por consola.
- Si llamas a Sumar(a, b), devuelve la suma de dos números.

Recursividad:

La mayoría de las funciones de Científica se llaman a sí mismas hasta llegar a un caso base:

- Factorial: Multiplica n por el factorial de n-1.
- Fibonacci: Suma los dos valores anteriores de la serie para obtener el actual.
- División: Se resta el divisor al dividendo repetidamente y cuenta cuántas veces pudo hacerlo antes de que el número fuera menor al divisor.

Es notable cómo el código construye una operación sobre otra, por ejemplo: En la suma recursiva, esto funciona en realidad como una multiplicación

Código 15: “Triangulo Sierpinski”

¿De qué trata el programa?

Es una aplicación diseñada para **generar y gestionar fractales**. Un fractal es básicamente una figura geométrica compleja que se repite a sí misma a diferentes escalas. El programa no solo resuelve estas figuras, sino que también tiene la capacidad de guardar los resultados y abrir archivos.

¿Qué hace exactamente?

El programa divide sus tareas en dos partes fundamentales:

1. **Cálculo matemático (fractales.h/cpp):** Esta es la parte "inteligente" donde se definen las fórmulas para crear las figuras geométricas y este realiza los cálculos necesarios para definir cómo se ve.
2. **Gestión de archivos ("mane_archivos.h"):** Como no quieres que el dibujo o los datos calculados desaparezcan al cerrar el programa, este módulo se encarga de guardarlos en tu computadora en varios formatos (.txt, .json, .xml, .csv).

Posibles dificultades

- **La complejidad de los fractales:** Lo más retador fue entender cómo programar la recursividad (el proceso donde la figura se repite dentro de sí misma). Un pequeño error en la fórmula de la figura

Códigos del segundo Parcial

ADT

Arreglo - Pila, Cola y Lista (P3 a la P5)

P3 - Análisis Lógico de la Pila (Stack)

1. Introducción

En esta práctica desarrollamos un sistema en C++ aplicando Programación Orientada a Objetos para modelar múltiples Estructuras de Datos. El proyecto utiliza polimorfismo mediante una clase abstracta base llamada EstructurasDatos. Este reporte se centrará exclusivamente en analizar la lógica y sintaxis de la clase derivada Pila, la cual opera bajo el principio fundamental de LIFO (Last In, First Out).

2. Declaración y Control de Memoria Estática

Para construir la clase Pila, implementamos memoria estática mediante un arreglo primitivo `int datos[MAX]`, donde la constante de capacidad máxima está definida estáticamente en 5 (`MAX = 5`).

El control lógico absoluto de esta estructura recae sobre la variable de rastreo `tope`. En el archivo de implementación, el constructor inicializa esta variable con la instrucción `tope = -1;`. Este valor negativo es el núcleo lógico inicial: indica matemáticamente que no hay ningún índice válido apuntando al arreglo, previniendo así que el programa lea datos basura de la memoria RAM.

3. Lógica Algorítmica de Operaciones Base

La clase encapsula las operaciones clásicas de una pila y las enlaza a los métodos sobrescritos `agregar()` y `quitar()` definidos por la interfaz base.

Inserción (push): El algoritmo primero valida que no exista un desbordamiento (Overflow) comprobando si `tope == MAX - 1`. Si hay memoria disponible, la operación se realiza en una sola línea de código optimizada: `datos[++tope] = valor;`. El operador de pre-incremento (`++tope`) es vital aquí; le indica al procesador que primero debe sumar 1 al índice actual.

- **Extracción (pop):** Para proteger el programa de un subdesbordamiento (Underflow), el método valida que la pila no esté vacía (`tope == -1`). La extracción utiliza la sintaxis: `datos[tope--]`. A diferencia de la inserción, aquí aplicamos un post-decremento: el programa accede y muestra el valor en el índice actual e, inmediatamente después, resta 1 al `tope`. Esto realiza un "borrado lógico", por lo que no necesitamos sobrescribir la celda con un cero; simplemente la ignoramos en futuras operaciones.

4. Conclusión

- La lógica de esta Pila es sumamente eficiente porque delega todo el peso algorítmico al manejo correcto de los operadores de incremento y decremento de C++. Al aplicar correctamente el encapsulamiento y la herencia, garantizamos que el puntero simulado (`tope`) jamás exceda los límites de nuestra memoria estática, resultando en una estructura robusta y libre de errores de segmentación.

P4- Análisis Lógico de la Cola (Queue)

1. Introducción

En esta práctica de Estructuras de Datos, construimos un programa basado en Programación Orientada a Objetos aplicando herencia y polimorfismo mediante la clase abstracta `EstructurasDatos`. Este reporte se enfoca específicamente en el análisis de la clase derivada `ColaConcierto`, la cual modela matemáticamente el comportamiento de un TDA tipo FIFO (First In, First Out / Primero en Entrar, Primero en Salir).

2. Variables de Control y Memoria

A diferencia de la pila que solo requiere un rastreador, la lógica de una cola lineal en un arreglo estático (`int boletos[MAX]` con `MAX = 5`) requiere el uso de dos punteros simulados (índices): `frente` y `final`.

En la implementación dentro de `EstructurasDatos.cpp`, el constructor inicializa ambas variables en `-1`. Esta es la sintaxis base para representar un estado de "ausencia de datos", evitando que el programa intente leer basura de la memoria

RAM. La cola se considera completamente vacía matemáticamente cuando `frente == -1`.

3. Lógica de Encolar (Inserción)

El método `formarFan(int numeroBoleto)` maneja la inserción de datos.

Primero, bloquea el desbordamiento (Overflow) si `estaLlena()` devuelve verdadero (cuando `final == MAX - 1`).

- Si es el primer elemento absoluto que entra a la cola (`frente == -1`), el algoritmo debe ajustar el índice `frente = 0` para tener un punto de partida válido para futuras extracciones.
- La sintaxis de inserción es compacta y optimizada: `boletos[++final] = numeroBoleto;`. Mediante el operador de pre-incremento en C++, el puntero `final` se desplaza a la siguiente celda libre y luego se deposita el valor.

4. Lógica de Desencolar (Extracción)

El método `dejarEntrar()` extrae los elementos evaluándolos siempre desde el índice `frente`.

Para evitar un subdesbordamiento (Underflow), aborta la operación si la cola ya está vacía.

Borrado Lógico: El elemento no se elimina de la memoria física del arreglo. En su lugar, aplicamos la instrucción `frente++`. Al mover el índice de inicio hacia adelante, el dato anterior queda lógicamente inaccesible y "eliminado" del flujo.

- **Reseteo del Arreglo:** La parte más crítica de esta sintaxis lineal es la condición `if (frente >= final)`. Si extrajimos al último fanático de la cola, la estructura quedaría inutilizable porque los índices estarían al límite. Por ello, reiniciamos el ciclo volviendo a asignar `-1` a `frente` y `final`.

5. Conclusión

La sintaxis de la clase `ColaConcierto` demuestra un manejo sólido de los índices de arreglos en C++ para simular el comportamiento FIFO. Aunque la implementación lineal mediante borrado lógico (`frente++`) funciona perfecto para este límite de 5 elementos, como área de mejora para el semestre, podríamos evolucionar esta misma lógica hacia una "Cola Circular" usando el operador módulo (%), lo que nos permitiría reutilizar los espacios de memoria que van quedando vacíos al principio sin tener que esperar a que la estructura se vacíe por completo.

P5- Análisis Lógico de la Lista

1. Introducción

En esta práctica de Estructuras de Datos, aplicamos los conceptos de herencia y polimorfismo derivando clases de la estructura abstracta EstructurasDatos. Este reporte se enfoca en el análisis de la clase Lista, un Tipo de Dato Abstracto (TDA) que, a diferencia de las pilas y colas, permite la inserción secuencial y la extracción aleatoria mediante índices.

2. Control de Memoria Estática

La arquitectura de nuestra lista se basa en memoria estática. En la definición de la clase, declaramos un arreglo `int elementos[MAXIMO]` con una constante `MAXIMO = 10`.

El control lógico de este arreglo recae exclusivamente en la variable `cantidad`. En el constructor, inicializamos `cantidad = 0`; Esta variable tiene un doble propósito fundamental:

Actúa como contador total de elementos para los métodos `tamano()`, `estaVacia()` y `estaLlena()`.

1. Funciona como el índice lógico ("puntero") que nos indica exactamente en qué celda de memoria debemos insertar el próximo dato.

3. Sintaxis de Inserción Secuencial

El método `insertar(int valor)` maneja la entrada de datos. Tras validar que el arreglo no esté lleno, la inserción se resuelve en una sola instrucción optimizada: `elementos[cantidad++] = valor`; Al usar el operador de post-incremento (`cantidad++`), el programa primero guarda el valor en el índice actual e inmediatamente después le suma 1 a la variable, dejándola lista para la siguiente inserción sin riesgo de sobrescribir datos.

4. Lógica de Eliminación y Corrimiento de Memoria

El método `eliminar(int posicion)` es el bloque algorítmico más complejo de esta clase, ya que extraer un elemento del medio de un arreglo estático genera un "hueco" en la memoria. Para solucionarlo, implementamos un algoritmo de corrimiento (desplazamiento):

- Primero, se valida que el índice solicitado por el usuario exista lógicamente (`posicion >= 0` y `< cantidad`).

Posteriormente, un ciclo `for` itera desde la posición a borrar hasta el penúltimo elemento válido (`cantidad - 1`).

La instrucción `elementos[i] = elementos[i + 1];` mueve físicamente cada dato una celda hacia la izquierda, aplastando el valor que queríamos eliminar y cerrando la brecha de memoria.

- Finalmente, se aplica un borrado lógico general restando uno al contador: `cantidad--;`.

5. Conclusión

La sintaxis empleada en la clase Lista nos demuestra las ventajas y desventajas de los arreglos estáticos. Mientras que la inserción es inmediata y tiene una complejidad temporal de $O(1)$ constante, la eliminación aleatoria requiere mover los datos en cascada mediante un ciclo for, lo que nos da una complejidad de $O(n)$ en el peor de los casos. Para proyectos futuros más robustos, esta lógica nos justifica la necesidad de migrar hacia Listas Simplemente Ligadas utilizando punteros y memoria dinámica.

Puntero - Pila, Cola y Lista (P6 a la P7Bis)

P6- Lógica y Sintaxis de la Pila

1. Introducción

En esta práctica, dimos el salto fundamental de la memoria estática a la memoria dinámica (Heap). Utilizando Programación Orientada a Objetos, implementamos una clase Pila que hereda de la interfaz abstracta EstructurasDatos. El objetivo de este reporte es analizar cómo la sintaxis de punteros en C++ nos permite modelar el comportamiento LIFO (Last In, First Out) sin las limitantes de un tamaño máximo predefinido.

2. Arquitectura del Nodo y Memoria

Para abandonar el uso de arreglos estáticos, definimos un struct Nodo privado dentro de la clase Pila.

El control absoluto de la estructura se maneja con un único puntero rastreador: `Nodo* tope`.

En el constructor de la clase, inicializamos `tope = nullptr`. Esta es la condición sintáctica absoluta que le indica al programa que la estructura está matemáticamente vacía.

- Una ventaja crítica de esta implementación es visible en el método `estaLlena()`. A diferencia de las pilas basadas en arreglos, aquí el método simplemente retorna false de manera incondicional. Al utilizar memoria dinámica, la pila solo se llena si el sistema operativo se queda físicamente sin memoria RAM.

3. Lógica de Inserción (apilar)

Nodo* nuNodo = new Nodo(); solicita dinámicamente un bloque de memoria para instanciar el nuevo registro.

nuNodo->dato = valor; asigna la carga útil ingresada por el usuario.

nuNodo->sig = tope; enlaza el nuevo nodo con el resto de la pila, empujando lógicamente a los elementos anteriores hacia abajo.

- tope = nuNodo; actualiza nuestro rastreador principal para que ahora apunte a este nuevo elemento superior.

4. Lógica de Extracción y Prevención de Fugas (desapilar)

El método desapilar(int& valor) extrae datos por referencia y maneja la recolección de basura manual, un paso obligatorio en C++.

Tras validar que la pila no esté vacía, creamos un puntero auxiliar: Nodo* ex = tope;. Esto "sostiene" el nodo superior para no perder su dirección de memoria.

Extraemos el dato y reconectamos el tope al siguiente elemento válido: tope = ex->sig;.

- La instrucción más importante de este método es delete ex;. Esto destruye físicamente el nodo extraído, devolviendo los bytes al sistema operativo y previniendo fugas de memoria (Memory Leaks).

5. Conclusión

La implementación de la Pila Dinámica demuestra el verdadero poder de los punteros en C++. La lógica descrita no solo optimiza el rendimiento general, sino que hace a la estructura completamente autónoma. Esta misma lógica de limpieza la aplicamos en el destructor ~Pila() iterando un ciclo while con desapilar, garantizando que al cerrarse el programa, la memoria quede impecable.

P7- Lógica y Sintaxis de la Cola

1. Introducción

En esta práctica de Estructuras de Datos, continuamos nuestro estudio de la memoria dinámica (Heap) implementando una Cola basada en punteros. Esta clase hereda de la interfaz abstracta EstructurasDatos. El objetivo de este breve reporte es analizar cómo la sintaxis de C++ nos permite modelar perfectamente el comportamiento FIFO (First In, First Out) sin las restricciones de capacidad que imponen los arreglos estáticos.

2. Arquitectura de Nodos y Doble Puntero

A diferencia de la pila dinámica que solo requiere un rastreador superior, la cola necesita monitorear ambos extremos para mantener la complejidad algorítmica en $O(1)$. En nuestra cabecera definimos un struct Nodo y declaramos dos punteros clave: prim (frente) y fin (final).

En el constructor, ambos se inicializan en estado nulo (nullptr). Matemáticamente, el programa sabe que la estructura está vacía simplemente evaluando si `prim == nullptr`. Asimismo, al trabajar con memoria dinámica, el método `estaLlena()` siempre retorna false, ya que la única limitante real es la memoria RAM del equipo.

3. Lógica de Inserción (insertar)

Para ingresar un nuevo elemento, el método solicita un bloque de memoria usando `new Nodo()`. La genialidad de esta sintaxis radica en su bifurcación lógica:

- Si la cola está vacía, el nuevo nodo asume el rol de ser el primer elemento absoluto: `prim = NuNodo;`

Si ya existen elementos formados, encadenamos el nuevo nodo al final de la fila apuntando desde el último elemento válido: `fin->sig = NuNodo;`

- Sin importar cuál de los dos casos ocurra, la función siempre termina actualizando nuestro rastreador trasero: `fin = NuNodo;`

4. Lógica de Extracción y Limpieza (retirar)

El método de extracción asegura el cumplimiento de la política FIFO operando exclusivamente sobre el puntero prim.

Tras guardar el dato por referencia, creamos un puntero auxiliar de retención: `Nodo* nEliminar = prim;`. Esto es vital para no perder la dirección de memoria al mover los enlaces.

Manejo del último nodo: Si resulta que `prim == fin`, significa que estamos extrayendo el único elemento que quedaba. El algoritmo resetea la cola devolviendo ambos punteros a nullptr. De lo contrario, simplemente desplaza el inicio al siguiente en la fila: `prim = prim->sig;`

- Finalmente, ejecutamos `delete nEliminar;` para destruir el nodo físicamente y devolverle esa memoria al sistema operativo.

5. Conclusión

La sintaxis implementada en la Cola Dinámica nos enseña a manipular referencias de memoria cruzadas (sig) y a gestionar múltiples rastreadores simultáneamente. Al asegurar el uso estricto de la instrucción delete tanto en la extracción como en el destructor de la clase (donde un ciclo while vacía la cola por completo), hemos construido un Tipo de Dato Abstracto profesional, rápido y libre de fugas de memoria (Memory Leaks).

P7BIS - Lógica y Sintaxis de la Lista

1. Introducción

En esta práctica de Estructuras de Datos, desarrollamos una Lista Dinámica implementando el uso de punteros y gestión de memoria en el Heap. La clase Lista hereda de nuestra interfaz abstracta EstructurasDatos. Este breve reporte analiza cómo la sintaxis de C++ nos permite enlazar, recorrer y eliminar nodos de forma secuencial sin las restricciones de capacidad impuestas por un arreglo estático.

2. Arquitectura del Nodo y el Puntero Inicial

Para gestionar la memoria, dentro de la definición de nuestra clase Lista declaramos un struct Nodo privado que contiene la variable entera dato y un puntero de autorreferencia Nodo* sig.

Toda la estructura está anclada a un único puntero rastreador principal llamado cabeza. En el constructor del archivo de implementación, inicializamos cabeza = nullptr;. Esta sintaxis es el pilar lógico que le indica al programa que la lista está matemáticamente vacía desde el momento en que se instancia.

3. Lógica de Inserción (insertarFin)

A diferencia de la pila que apila datos al inicio, nuestra lista está diseñada para anexar los nuevos elementos al final de la cadena.

Para lograr esto, creamos un nuevo bloque de memoria con `Nodo* NuNodo = new Nodo();` y le asignamos su valor.

Si la estructura está completamente vacía, la inserción es directa: `cabeza = NuNodo;`.

Si la lista ya contiene datos, aplicamos un recorrido lógico. Instanciamos un puntero auxiliar `Nodo* actual = cabeza;` y utilizamos un ciclo while (`actual->sig != nullptr`). Este ciclo avanza de nodo en nodo hasta detectar el final de la cadena. Al detenerse en el último nodo válido, lo enlazamos al recién creado con la instrucción `actual->sig = NuNodo;`.

4. Lógica de Búsqueda y Extracción

(eliminarEspecifico)

El método de eliminación es el más complejo a nivel sintáctico porque opera basándose en la coincidencia de un valor, no en índices fijos.

Caso Base (Inicio): Si el dato buscado está en la primera posición (`cabeza->dato == n`), creamos un puntero auxiliar de respaldo (`ex`), adelantamos la cabeza al segundo

elemento con cabeza = cabeza->sig; y aplicamos delete ex; para destruir el elemento inicial.

Caso General (Medio o Final): Si el dato está más adelante, usamos un ciclo que "mira al futuro" evaluando el dato del nodo contiguo: while (actual->sig != nullptr && actual->sig->dato != n).

- Al detenernos exactamente un nodo antes del objetivo, "puenteamos" la conexión para desenlazarlo del flujo lógico de la lista (actual->sig = ex->sig;) y procedemos a destruirlo físicamente de la memoria RAM con delete ex;.

5. Conclusión

La sintaxis de la Lista Dinámica nos demuestra la flexibilidad del paradigma de nodos entrelazados en C++. Dominar la instrucción actual = actual->sig para iterar elementos es un paso crítico en nuestra formación profesional. Asimismo, asegurarnos de utilizar delete durante las eliminaciones manuales y dentro del destructor de la clase (el cual vacía la lista por completo al cerrar el programa), nos garantiza crear sistemas totalmente libres de fugas de memoria (Memory Leaks).

Librerías - Pila, Cola y Lista (P8 a la P10)

P8 - Lógica y Sintaxis de la Pila

1. Introducción

En esta práctica dimos un paso muy importante en la materia de Estructuras de Datos al dejar atrás la gestión manual de memoria y comenzar a utilizar la Standard Template Library (STL) de C++. Este breve reporte se enfoca en analizar la clase Navegador, la cual simula el historial de navegación web empleando una Pila nativa bajo el principio LIFO (Last In, First Out).

2. Declaración y Abstracción de Memoria

Para tener acceso a esta estructura, incluimos la directiva #include en nuestro archivo de cabecera. Dentro de la definición de la clase Navegador, la estructura se declara simplemente como stack<string> pilaHistorial;.

A nivel de tercer semestre, esto representa una abstracción enorme: la STL se encarga por completo de crear los nodos subyacentes y gestionar la memoria dinámica en el Heap. Ya no necesitamos preocuparnos por asignar memoria con new ni liberarla con delete, evitando así posibles fugas de memoria.

3. Lógica Algorítmica: Inserción y Extracción

La sintaxis para manipular nuestra pila se simplifica utilizando los métodos nativos de la librería en el archivo de implementación:

Inserción (push): Cuando el usuario visita una nueva página, el método `Pagina` encapsula la instrucción `pilaHistorial.push(dir);`. Esto empuja la URL (cadena de texto) automáticamente a la cima de la estructura.

Consulta y Extracción (pop y top): Para simular el botón de "Atrás" en el navegador, el método `Anterior` primero utiliza `pilaHistorial.empty()` para proteger el programa de un `Underflow` (intentar borrar algo que no existe). Si hay historial, consultamos el valor superior con `pilaHistorial.top()` y procedemos a eliminarlo definitivamente de la memoria ejecutando `pilaHistorial.pop();`.

4. Visualización y el Problema de la Iteración

A diferencia de los arreglos o las listas, la clase `stack` de la STL es muy estricta y no proporciona iteradores; es decir, no podemos usar un ciclo `for` para leer todos los datos desde el tope hasta el fondo.

Para resolver nuestro método `mostrar()` sin perder datos, aplicamos una solución algorítmica específica: instanciamos una pila auxiliar haciendo una copia directa de la original mediante `stack copiaHistorial = pilaHistorial;`. Posteriormente, usamos un ciclo `while` para extraer (`.pop()`) e imprimir (`.top()`) todos los elementos de la pila temporal, dejando nuestro historial real intacto para seguir operando.

5. Conclusión

Implementar una pila mediante la librería nos demuestra cómo se desarrolla el software en entornos profesionales. Aunque haber aprendido a enlazar punteros manualmente fue vital para nuestra lógica matemática, utilizar la STL reduce drásticamente las líneas de código, elimina los errores de segmentación y nos permite centrarnos al 100% en la arquitectura y reglas de negocio de nuestra aplicación (como en este caso, el comportamiento del historial de navegación).

P9 - Lógica y Sintaxis de la Cola

1. Introducción

En esta práctica dimos un avance muy importante en la materia de Estructuras de Datos al sustituir la gestión manual de punteros por la `Standard Template Library (STL)` de C++. Este breve reporte analiza específicamente la clase `Turnos`, la cual modela un sistema de atención a clientes apoyándose en una estructura de Cola bajo el principio `FIFO (First In, First Out)`.

2. Declaración y Abstracción

Para poder hacer uso de la cola nativa de C++, agregamos la directiva `#include` en nuestro archivo de cabecera `EstructurasConLibrerias.h`. En esa misma clase, instanciamos nuestra estructura con la instrucción `queue filaC`;

Como estudiantes de tercer semestre, esto representa una abstracción valiosa: la STL se encarga en segundo plano de gestionar los punteros de frente y final, así como de solicitar la memoria dinámica mediante nodos. Nuestra única preocupación ahora es la lógica del negocio.

3. Lógica Algorítmica: Inserción y Extracción

La manipulación de los datos se realiza en el archivo `EstructurasConLibrerias.cpp` utilizando los métodos nativos, lo que reduce la posibilidad de cometer errores de memoria:

Encolar (push): En el método `registrarC`, usamos `filaC.push(nombre)`; Esto automáticamente reserva el espacio e inserta al nuevo cliente al final de la estructura, operando en tiempo constante $O(1)$.

- Consultar y Desencolar (front y pop): El método `atenderSig` controla la salida. Para evitar un Underflow, primero valida la existencia de datos con `filaC.empty()`. Si la cola no está vacía, extraemos el valor delantero con `filaC.front()` y lo destruimos de la memoria llamando a `filaC.pop()`;

4. El Reto de la Iteración (mostrar)

Una característica estricta de `queue` en la STL es que no posee iteradores; no podemos usar un ciclo `for` tradicional para imprimir a los clientes porque romperíamos la regla matemática del encapsulamiento FIFO.

Para solventar esto en nuestro método `mostrar`, aplicamos una estrategia de clonación: instanciamos una cola temporal copiando la original mediante `queue<string> copiaFila = filaC`. Posteriormente, usamos un ciclo `while` (`copiaFila.empty()`) para leer el frente e ir destruyendo (`.pop()`) los elementos de la copia. Así logramos imprimir toda la fila en pantalla sin alterar los datos de la cola real con la que estamos trabajando.

5. Conclusión

El uso de `<queue>` simplifica drásticamente nuestro código. Evitamos el manejo explícito de variables temporales, condicionales complejas o las sentencias `new` y `delete`. Esto nos permite programar de manera más segura y con estándares industriales, garantizando un código limpio y libre de fugas de memoria.

P10 - Lógica y Sintaxis de la Lista

1. Introducción

En esta práctica consolidamos nuestros conocimientos de Estructuras de Datos dando el salto a la Standard Template Library (STL) de C++. Este breve reporte se enfoca en el análisis de la clase `Inventario`, la cual gestiona objetos de tipo `Producto` utilizando la estructura dinámica `std::list`.

2. Declaración y Abstracción de Memoria

Para implementar esta estructura, primero agregamos la directiva `#include <list>` en el archivo de cabecera. Al declarar la variable privada `list obInv`, la STL abstrae por completo la gestión de la memoria dinámica. Como estudiantes de tercer semestre, sabemos que internamente esto opera como una Lista Doblemente Ligada, pero la librería nos ahorra la tarea de crear manualmente estructuras `Nodo` y de gestionar los punteros entrelazados, evitando así el riesgo de errores de segmentación.

3. Lógica de Inserción y Extracción

En el archivo de implementación, la manipulación de datos se resuelve de forma directa y segura con los métodos nativos del contenedor:

1. Inserción (`push_back`): El método `agregar()` recopila el código, nombre y cantidad, instancia un objeto `Producto` y lo inserta directamente al final de la estructura usando `obInv.push_back(...)`. El sistema operativo maneja la asignación de memoria en tiempo $O(1)$.
- Extracción (`pop_back` y `back`): Para la función `quitar()`, protegemos el flujo verificando primero `!obInv.empty()`. Para avisar qué producto se eliminará, accedemos al atributo del último elemento con `obInv.back().nombre` y, en la línea siguiente, lo destruimos de la memoria llamando a `obInv.pop_back()`.

4. El Uso Avanzado de Iteradores (mostrar)

La gran ventaja de la clase `list` frente a `stack` o `queue` de la STL es que sí permite recorridos secuenciales sin destruir los datos. Para nuestro método `mostrar()`, implementamos un iterador nativo en lugar de copias temporales.

La sintaxis del ciclo `for` es clave aquí:

```
for(list<Producto>::iterator it = obInv.begin(); it != obInv.end(); ++it)
```

Declaramos un "puntero inteligente" (`it`) que arranca en la primera dirección válida (`begin()`) y avanza lógicamente nodo por nodo (`++it`) hasta que alcanza la marca del final (`end()`). Para invocar la información dentro de cada nodo, utilizamos el operador flecha: `it->mostrarDet()`.

5. Conclusión

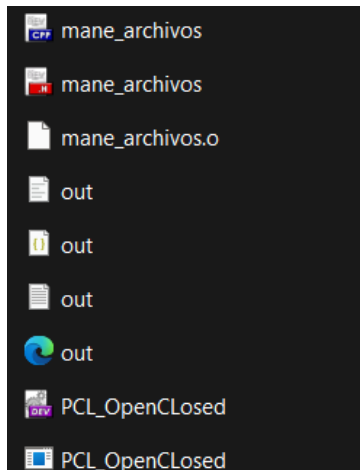
Sustituir nuestras listas dinámicas manuales por la librería `<list>` marca un punto de inflexión en nuestra formación. Nos libera de gestionar instrucciones `new` y `delete`, y nos ofrece herramientas poderosas y seguras como los iteradores. Esto nos permite centrarnos completamente en la lógica de negocio —en este caso, un inventario de productos— generando un código más limpio, fácil de leer y libre de fugas de memoria.

ARCHIVOS

1. Introducción

El objetivo de esta fase del proyecto fue integrar una nueva clase que fue dedicado exclusivamente a la persistencia de datos, permitiendo que la información manipulada en el sistema no se pierda al finalizar la ejecución, en pocas palabras, no solo se visualice en la consola del programa, si no, que se imprima en lo que son los siguientes tipos de archivo, como lo que es el **txt**, **xml**, **csv** y **json**. Para ello, se creó el archivo **"mane_archivos.h"**, el cual gestiono toda la parte de la lectura y escritura en los archivos de texto.

Este archivo se ocupó tanto en los códigos del primer parcial, como del segundo.



```
n.cpp EstructurasDatos.h EstructurasDatos.cpp [1] mane_archivos.h
    else if (c == '<') escaped += "<";
    else if (c == '>') escaped += ">";
    else if (c == '"') escaped += """;
    else if (c == '\\') escaped += "\\&apos;";
    else escaped += c;
}
ofstream f(path.c_str(), std::ios::trunc);
f << "<?xml version='1.0' encoding='UTF-8'?'>\n"
  << "<programas>\n"
  << "<entrada>\n"
  << "<programa>" << programa << "</programa>\n"
  << "<salida>" << escaped << "</salida>\n"
  << "</entrada>\n"
  << "</programas>\n";
}

inline void writeCSV(const string& programa, const string& salida)
{
    string path = std::string(OUTPUT_DIR) + "out.csv";
    string escaped;
    for (size_t i = 0; i < salida.length(); ++i)
    {
        char c = salida[i];
        escaped += (c == '"') ? string("\\&apos;") : string(1, c);
    }
    ofstream f(path.c_str(), std::ios::trunc);
    f << "programa,salida\n"
      << "\"" << programa << "\",\"" << escaped << "\"\n";
}

inline void writeTXT(const string& programa, const string& salida)
{
    string path = string(OUTPUT_DIR) + "out.txt";
    ofstream f(path.c_str(), std::ios::trunc);
    f << "PROGRAMA: " << programa << "\n"
      << "SALIDA: " << salida << "\n";
}

inline void writeAllOutputs(const string& programa, const string& salida)
{
    writeJSON(programa, salida);
    writeCSV(programa, salida);
    writeTXT(programa, salida);
}
```

2. Descripción del “mane_archivos”.

Este módulo actúa como una capa de abstracción para el manejo de archivos. Su estructura permite que el resto del programa (main.cpp) no necesite conocer los detalles de bajo nivel sobre cómo se abren o cierran los flujos de datos.

Funcionalidades:

- **Recuperación:** Se incluyeron rutinas para cargar la información almacenada en disco de vuelta a la memoria del programa al momento de iniciar el sistema.
- Se implementó un método que recorre las estructuras en memoria y las escribe en el archivo mediante el flujo “**std::ofstream**”. Esto permite convertir los objetos en el programa a un formato de texto estructurado.
- Se desarrolló una rutina de carga que utiliza “**std::ifstream**” para leer el archivo línea por línea o registro por registro, reconstruyendo los objetos en memoria para que el programa pueda trabajar con ellos al iniciar.
- Se ocupó el `_open()` antes de cualquier operación. Esto evita que el programa colapse si el archivo de datos no se encuentra, o si el archivo está siendo utilizado por otro proceso del sistema.
- **Limpieza de Buffer y Cierre de Seguridad:** Cada función de escritura incluye una limpieza explícita del buffer y el cierre del archivo.

3. Uso de `system(start...)`

Para la visualización de los reportes generados, implementé la función `system("start...")`. Este comando aprovecha la asociación de archivos del sistema operativo Windows para invocar el programa predeterminado del usuario para cada extensión específica (por ejemplo, el Bloc de notas para `.txt`, Excel para `.csv`, o visores de código para `.json` y `.xml`).

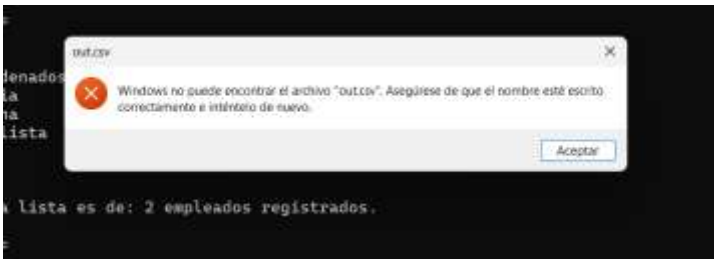
Este comando hace que, al momento de ejecutar el programa, no solo se visualice en la consola, sino que abre automáticamente el documento generado en el entorno de trabajo del usuario, facilitando la validación inmediata de la información procesada.

```
system("start out.txt");  
system("start out.json");  
system("start out.xml");  
system("start out.csv");
```

4. Análisis de Errores

Durante la implementación de este módulo, tuve algunos errores que pensé que no tenían solución:

1. Al ejecutar el programa y escribir lo que me pedía el programa, al finalizar se supone que se deben de imprimir en los archivos de texto, pero no fue así, ya que aún no se habían creado, por ende, me aparecía el siguiente error:



```
7. Salir
Elige una opción: 7
Saliendo del programa...
El sistema no puede encontrar el archivo out.txt.
El sistema no puede encontrar el archivo out.json.
```

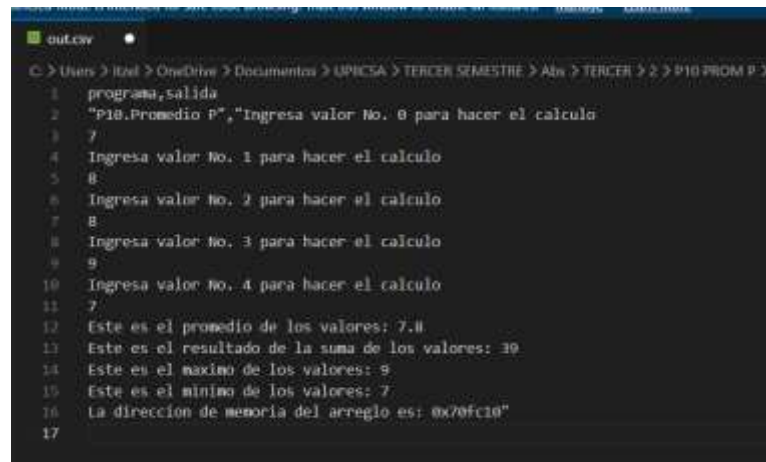
Así me apareció con todos los programas donde implemente los archivos de texto, lo único que hice fue darle a “Reconstruir todo” y “Compilar y Ejecutar”, volví a llenar todo lo que me solicitaba el programa y así como elegía la opción de salir, automáticamente se abrían los archivos de texto (txt, xml, csv y Json) y con eso logre visualizar lo que había escrito en la consola del sistema.

2. El otro problema que tuve es, que al momento de escribir en la consola, el los archivos de texto no aparecía lo que el usuario había escrito (en este caso, lo que yo había escrito), lo que se me hizo raro y el programa compilada bien, después tuve que agregar un **“cout << nombre de la variable << “\n”;** “ después de pedirla, para que esta me la imprimiera y así logre que se visualizara en los archivos de texto, aunque en la consola se repetía, pero fue la forma fácil que encontré para que se pudiera observar.

```
Menu de Cola (Queue)
1. Agregar elemento
2. Quitar elemento
3. Mostrar contenido
4. Verificar si esta vacia
5. Verificar si esta llena
6. Mostrar tamano
7. Regresar al menu principal
Elige una opcion: 1
1
Ingresa el numero de boleto para la cola: 23
23
Boleto #23 formado en la cola.
```

```
out.json out.csv
C:\Users\itziel > OneDrive > Documentos > UPIICSA > TERCER SEMESTRE
20
21 MENU DE ESTRUCTURAS
22 1. Trabajar con Pila (stack)
23 2. Trabajar con cola (Queue)
24 3. Trabajar con Lista
25 4. Salir
26 Elige una opcion: 2
27
28 Menu de Cola (Queue)
29 1. Agregar elemento
30 2. Quitar elemento
31 3. Mostrar contenido
32 4. Verificar si esta vacia
33 5. Verificar si esta llena
34 6. Mostrar tamano
35 7. Regresar al menu principal
36 Elige una opcion: 1
37 Ingresa el numero de boleto para la cola: 23
38 Boleto #23 formado en la cola.
39
40 Menu de Cola (Queue)
41 1. Agregar elemento
42 2. Quitar elemento
43 3. Mostrar contenido
44 4. Verificar si esta vacia
45 5. Verificar si esta llena
46 6. Mostrar tamano
47 7. Regresar al menu principal
48 Elige una opcion: 7
49 Regresando...
```


3. También tuve algunos errores al implementar la estructura del archivo "mane_archivos.h", pero al final se logro
4. **Gestión de Flujos:** Tuve problemas con archivos bloqueados. Aprendí que siempre debo llamar a .close() después de cada operación de lectura o escritura para liberar los recursos del sistema operativo.



```
C:\Users\Itzel > OneDrive > Documentos > UPICSA > TERCER SEMESTRE > Abs > TERCER > 2 > P10 FROM P >
1  programa_salida
2  "P10.Promedio P", "Ingresa valor No. 0 para hacer el calculo
3  7
4  Ingresa valor No. 1 para hacer el calculo
5  8
6  Ingresa valor No. 2 para hacer el calculo
7  8
8  Ingresa valor No. 3 para hacer el calculo
9  9
10 Ingresa valor No. 4 para hacer el calculo
11 7
12 Este es el promedio de los valores: 7.8
13 Este es el resultado de la suma de los valores: 39
14 Este es el maximo de los valores: 9
15 Este es el minimo de los valores: 7
16 La dirección de memoria del arreglo es: 0x70fc10"
17
```

Escribir y leer CSV datos básicos, arreglos, creados arreglos y los ADT

Introducción Esta práctica demuestra la aplicación de conceptos fundamentales de abstracción de datos mediante la creación de un sistema de registro. El objetivo principal es encapsular la información de estudiantes (nombre, edad y un historial dinámico de calificaciones) en un Tipo de Dato Abstracto (TDA) y lograr que estos datos persistan en el tiempo utilizando un archivo de texto con formato CSV (Valores Separados por Comas).

¿Cómo Funciona el Programa? El sistema está diseñado para ser seguro e interactivo, dividiendo su funcionamiento en tres pilares clave:

1.- El Modelo de Datos: Se implementó la clase Alumno para agrupar atributos de distintos tipos. Las calificaciones se almacenan utilizando un std::vector, lo que permite que cada estudiante tenga un número diferente de notas (o ninguna) sin desperdiciar memoria RAM.

```

7 class Alumno {
8 public:
9     std::string nombre;
10    int edad;
11    std::vector<int> calificaciones;
12    Alumno();
13    Alumno(std::string n, int e, std::vector<int> c);
14    void mostrar() const;
15 };
16
17 void guardarEnCSV(const std::string& nombreArchivo, const std::vector<Alumno>& listaAlumnos);
18 std::vector<Alumno> leerDesdeCSV(const std::string& nombreArchivo);

```

2.- Interfaz Segura: El usuario controla el flujo mediante un menú en consola. Para evitar "crasheos", se utilizan funciones de limpieza de buffer (cin.fail(), cin.clear(), cin.ignore()). Esto garantiza que si se ingresa una letra accidentalmente donde se espera la edad o una calificación, el programa maneja el error y solicita el dato nuevamente.

```

C:\Users\gabri\OneDrive\Desktop
MENU DE ALUMNOS
1. Agregar un nuevo alumno
2. Guardar datos en CSV
3. Cargar y mostrar datos desde CSV
4. Salir
Elige una opción: 1

AGREGAR ALUMNO
Nombre del alumno: Omar
Edad: 21
Ingresa las calificaciones (escribe -1 para terminar):
Nota: 7
Nota: 9
Nota: 5
Nota: 6
Nota: -1

Alumno 'Omar ' agregado al sistema.
MENU DE ALUMNOS
1. Agregar un nuevo alumno
2. Guardar datos en CSV
3. Cargar y mostrar datos desde CSV
4. Salir
Elige una opción: 2

GUARDANDO EN CSV
Datos guardados exitosamente en alumnos.csv

```

3.- Persistencia (Lectura y Escritura): Al guardar los datos con , los atributos principales se separan por comas (,), pero el vector de calificaciones utiliza puntos y comas (;) de manera interna. Esto previene que el CSV confunda una calificación con una nueva columna. Al cargar los datos, la herramienta std::stringstream escanea cada línea de texto y la "recorta" usando estos delimitadores para reconstruir los objetos en C++.

Nombre	Edad	Calificaciones
Omar	21	7;9;5;6

Conclusión El desarrollo de este programa comprueba que la abstracción de datos es esencial no solo para organizar código en memoria, sino para traducir estructuras dinámicas a formatos de almacenamiento plano. La implementación de delimitadores personalizados para arreglos internos y un manejo defensivo de los errores de entrada del usuario dan como resultado un sistema robusto, funcional y escalable.

Escribir y leer XML datos básicos, arreglos, creados arreglos y los ADT.

Introducción En esta práctica se aborda la abstracción de datos enfocada en la serialización estructurada mediante XML (Lenguaje de Marcado Extensible). El propósito es gestionar una colección de videojuegos, registrando su título, horas de juego y un historial dinámico de puntajes. A diferencia de los archivos planos, el XML permite almacenar esta información en una jerarquía de etiquetas, lo que facilita tanto la lectura humana como la validación de la estructura del Tipo de Dato Abstracto (TDA).

¿Cómo Funciona el Programa? El sistema se divide en tres componentes principales:

1.- El Modelo de Datos: La clase Videojuego agrupa las propiedades del objeto. Se destaca el uso de un contenedor dinámico `std::vector` para los puntajes, permitiendo que cada juego registre múltiples partidas (etiquetas hijas) dentro de una etiqueta constructora padre ().

```
7 class Videojuego {
8     public:
9         string titulo;
10        int horasJugadas;
11        vector<int> puntajesPartidas;
12        Videojuego();
13        Videojuego(string t, int h, vector<int> p);
14        void mostrar() const;
15    };
16
17    void guardarEnXML(const string& nombreArchivo, const vector<Videojuego>& catalogo);
18    vector<Videojuego> leerDesdeXML(const string& nombreArchivo);
19
20    string extraerContenidoEtiqueta(const string& linea, const string& etiqueta);
```

2.- Interfaz Interactiva y Segura: Implementada en `main.cpp`, cuenta con un menú cíclico que guía al usuario. Se incluyeron validaciones defensivas usando `std::cin.fail()` y `std::cin.ignore()`. Si el usuario ingresa texto por error al solicitar las horas jugadas o un puntaje, el programa limpia el buffer de entrada, cancela el registro defectuoso y evita que la consola colapse, garantizando la integridad de la sesión.

```
C:\Users\gabriel\OneDrive\Documents
BIBLIOTECA DE VIDEOJUEGOS
1. Agregar un nuevo videojuego
2. Guardar datos en XML
3. Cargar y mostrar datos desde XML
4. Salir
Elige una opción: 1

AGREGAR VIDEOJUEGO
Titulo del juego: Minecraft
Horas jugadas: 100
Ingresa los puntajes de las partidas (escribe -1 para terminar):
Puntaje: 4000
Puntaje: -1

Juego 'Minecraft' agregado correctamente a la biblioteca.
BIBLIOTECA DE VIDEOJUEGOS
1. Agregar un nuevo videojuego
2. Guardar datos en XML
3. Cargar y mostrar datos desde XML
4. Salir
Elige una opción: 1

AGREGAR VIDEOJUEGO
Titulo del juego: Fortnite
Horas jugadas: 50
Ingresa los puntajes de las partidas (escribe -1 para terminar):
Puntaje: 9999
Puntaje: -1

Juego 'Fortnite' agregado correctamente a la biblioteca.
```

3.- Persistencia y "Parsing" (Lectura/Escritura XML): La escritura se logra estructurando manualmente las etiquetas (ej. ...) usando la librería . Para la lectura, se desarrolló el algoritmo personalizado `extraerContenidoEtiqueta`. Este método escanea cada línea del archivo, localiza la posición matemática de la etiqueta de apertura (<) y la de cierre (</>), y extrae únicamente la subcadena (substr) de texto o números contenida en el medio para reconstruir el objeto.

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8" />
<LibreriaJuegos>
  <Videojuego>
    <Titulo>Minecraft</Titulo>
    <HorasJugadas>100</HorasJugadas>
    <Partidas>
      <Puntaje>4000</Puntaje>
    </Partidas>
  </Videojuego>
  <Videojuego>
    <Titulo>Fortnite</Titulo>
    <HorasJugadas>50</HorasJugadas>
    <Partidas>
      <Puntaje>9999</Puntaje>
    </Partidas>
  </Videojuego>
</LibreriaJuegos>
```

Conclusión: El desarrollo de este programa demuestra que la abstracción de datos va de la mano con la manipulación avanzada de cadenas de texto (std::string). Crear un algoritmo de "parsing" manual para extraer datos de etiquetas XML refuerza el entendimiento de cómo las aplicaciones reales interpretan jerarquías de información. El resultado es un código escalable y resistente a errores de usuario.

Escribir y leer txt datos básicos, arreglos, creados arreglos y los ADT

Introducción Esta práctica ilustra la serialización de datos utilizando archivos de texto plano (TXT). El objetivo es gestionar un inventario de hardware computacional, encapsulando en un Tipo de Dato Abstracto (TDA) el procesador, la capacidad de

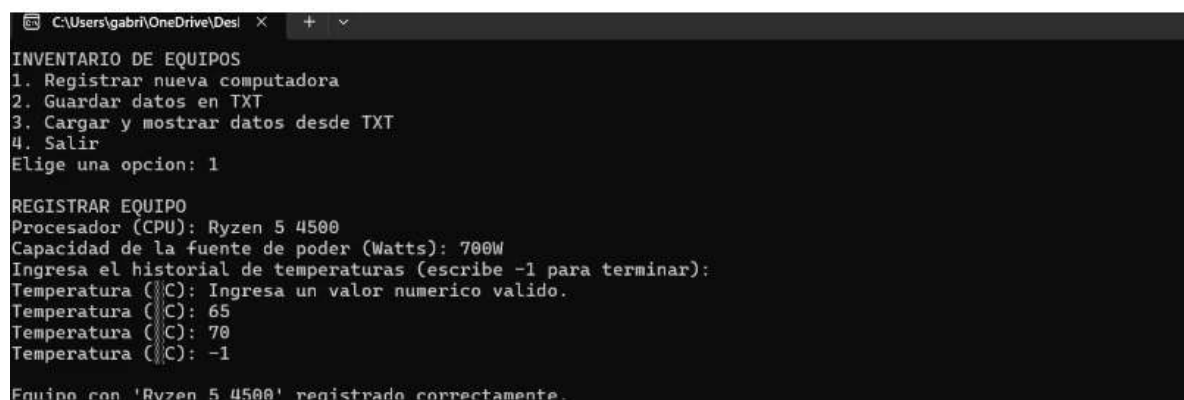
la fuente de poder (en watts) y un historial dinámico de temperaturas. A diferencia de formatos estructurados como CSV o XML, el uso de TXT exige diseñar un protocolo de lectura y escritura personalizado basado en saltos de línea y separadores visuales para mantener la integridad de los registros.

¿Cómo Funciona el Programa? El sistema garantiza el manejo seguro y persistente de los datos a través de los siguientes pilares:

1.- El Modelo de Datos: Se creó la clase Computadora para unificar variables de distintos tipos (string e int). El historial térmico se almacena en un `std::vector`, lo que permite registrar desde una sola lectura hasta múltiples picos de temperatura de manera dinámica.

```
7 class Computadora {
8     public:
9         string procesador;
10        int wattsFuente;
11        vector<int> historialTemperaturas;
12        Computadora();
13        Computadora(string proc, int watts, vector<int> temps);
14        void mostrar() const;
15    };
16
17    void guardarEnTXT(const string& nombreArchivo, const vector<Computadora>& inventario);
18    vector<Computadora> leerDesdeTXT(const string& nombreArchivo);
```

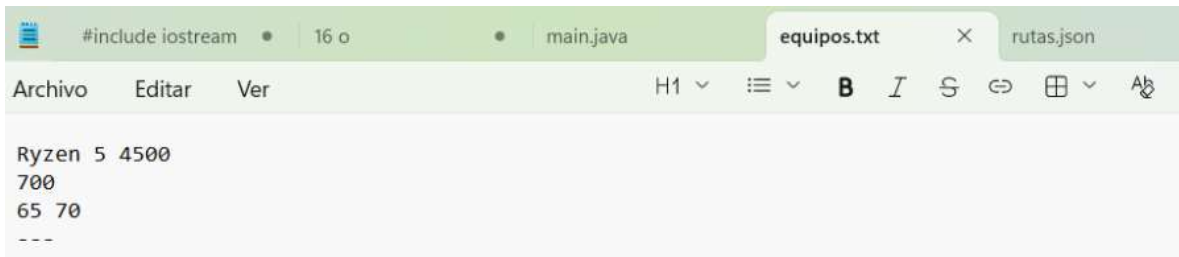
2.- Interfaz Interactiva y Segura: El programa cuenta con un menú en consola que utiliza un ciclo continuo. Integra limpieza de buffer mediante `cin.ignore(numeric_limits::max(), '\n')`. Esta validación es crucial para evitar errores de lectura cuando el usuario alterna entre capturar texto con espacios (con `getline` para el CPU) y capturar números (para los watts y las temperaturas).



```
C:\Users\gabri\OneDrive\Desl x + v
INVENTARIO DE EQUIPOS
1. Registrar nueva computadora
2. Guardar datos en TXT
3. Cargar y mostrar datos desde TXT
4. Salir
Elige una opcion: 1

REGISTRAR EQUIPO
Procesador (CPU): Ryzen 5 4500
Capacidad de la fuente de poder (Watts): 700W
Ingresa el historial de temperaturas (escribe -1 para terminar):
Temperatura (C): Ingresa un valor numerico valido.
Temperatura (C): 65
Temperatura (C): 70
Temperatura (C): -1
Equipo con 'Ryzen 5 4500' registrado correctamente
```

3.- Persistencia (Lectura/Escritura Secuencial): La exportación a TXT guarda cada atributo en una línea nueva, separando las temperaturas con espacios simples y dividiendo cada registro (cada computadora) con el marcador ---. Durante la carga, el sistema utiliza `getline` secuencialmente. Cuando lee la línea de temperaturas,



```
#include iostream
16 o
main.java
equipos.txt
rutas.json

Archivo  Editar  Ver  H1  B  I  S  ⇌  ⌵  A

Ryzen 5 4500
700
65 70
---
```

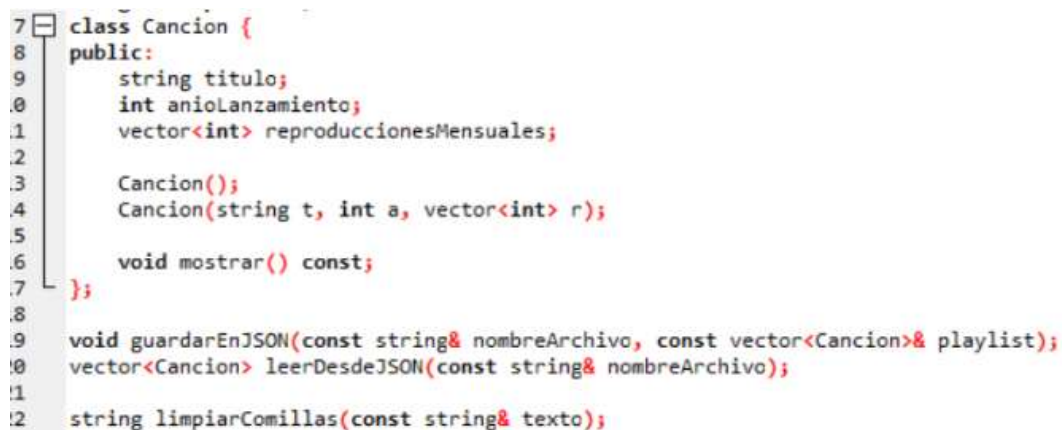
Conclusión: La manipulación de archivos TXT demuestra que la abstracción de datos requiere un control total sobre el flujo de lectura y escritura. Al carecer de etiquetas nativas o delimitadores de columnas, el programador debe establecer una estructura secuencial estricta. El uso combinado de saltos de línea lógicos, separadores visuales y flujos de cadena (stringstream) resultó en un programa altamente eficiente y resistente a fallos de sintaxis.

Escribir y leer Json datos básicos, arreglos, creados arreglos y los ADT

Introducción Esta práctica aplica la abstracción de datos para gestionar un catálogo musical, implementando la serialización de la información en formato JSON (JavaScript Object Notation). El objetivo es almacenar un arreglo de objetos que representan canciones (con su título, año de lanzamiento y un registro dinámico de reproducciones mensuales). JSON es el estándar actual para el intercambio de datos en aplicaciones modernas, por lo que generar y leer su estructura de llaves { }, corchetes [] y comillas de forma manual refuerza el dominio sobre el manejo avanzado de cadenas.

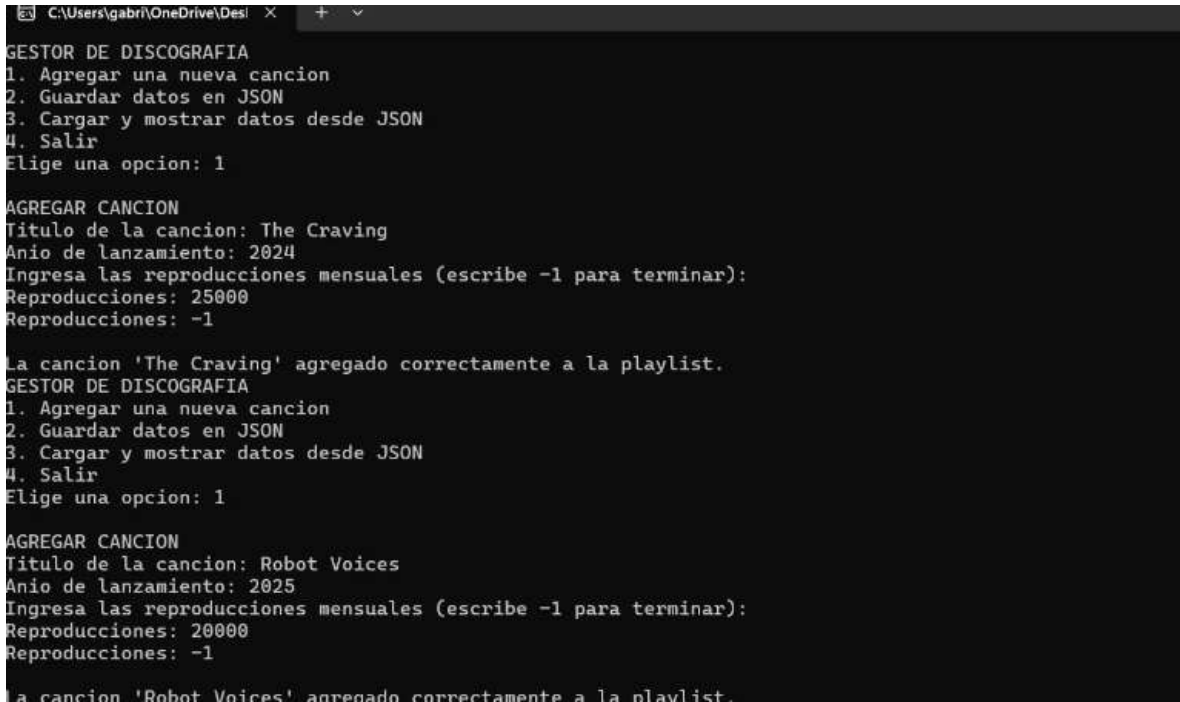
¿Cómo Funciona el Programa? El sistema divide el problema en tres secciones clave:

1.- El Modelo de Datos: Se definió el TDA mediante la clase Cancion. El uso de std::vector para las reproducciones mensuales permite almacenar el rendimiento del track a lo largo del tiempo de manera dinámica, sin limitar la cantidad de meses registrados por cada objeto.



```
7 class Cancion {
8     public:
9         string titulo;
10        int anioLanzamiento;
11        vector<int> reproduccionesMensuales;
12
13        Cancion();
14        Cancion(string t, int a, vector<int> r);
15
16        void mostrar() const;
17    };
18
19    void guardarEnJSON(const string& nombreArchivo, const vector<Cancion>& playlist);
20    vector<Cancion> leerDesdeJSON(const string& nombreArchivo);
21
22    string limpiarComillas(const string& texto);
```


2.- Interfaz Segura: El bloque principal (main.cpp) emplea un bucle interactivo protegido. El uso sistemático de `cin.fail()`, `cin.clear()` y `cin.ignore()` blindo al programa contra interrupciones abruptas (por ejemplo, si se introduce texto al solicitar el año de lanzamiento de la canción), ofreciendo una experiencia de usuario ininterrumpida y permitiendo agregar múltiples registros mediante una bandera de parada (-1).



```
GESTOR DE DISCOGRAFIA
1. Agregar una nueva cancion
2. Guardar datos en JSON
3. Cargar y mostrar datos desde JSON
4. Salir
Elige una opcion: 1

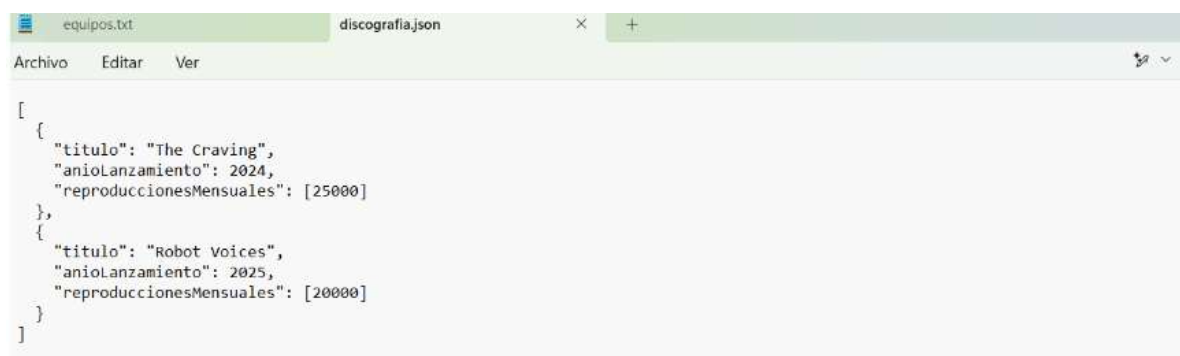
AGREGAR CANCION
Titulo de la cancion: The Craving
Año de lanzamiento: 2024
Ingresa las reproducciones mensuales (escribe -1 para terminar):
Reproducciones: 25000
Reproducciones: -1

La cancion 'The Craving' agregado correctamente a la playlist.
GESTOR DE DISCOGRAFIA
1. Agregar una nueva cancion
2. Guardar datos en JSON
3. Cargar y mostrar datos desde JSON
4. Salir
Elige una opcion: 1

AGREGAR CANCION
Titulo de la cancion: Robot Voices
Año de lanzamiento: 2025
Ingresa las reproducciones mensuales (escribe -1 para terminar):
Reproducciones: 20000
Reproducciones: -1

La cancion 'Robot Voices' agregado correctamente a la playlist.
```

3.- Serialización y Análisis ("Parsing") JSON: La escritura a disco se realiza imprimiendo cuidadosamente la sintaxis estricta del JSON (tabulaciones, comillas dobles escapadas `\` y separación por comas). Para la lectura, se implementó el algoritmo `limpiarComillas`, el cual extrae el valor puro de las cadenas. Además, el programa combina (para borrar comas residuales con `std::remove`) y `std::stringstream` para diseccionar el arreglo interno de reproducciones.



```
[
  {
    "titulo": "The Craving",
    "añoLanzamiento": 2024,
    "reproduccionesMensuales": [25000]
  },
  {
    "titulo": "Robot Voices",
    "añoLanzamiento": 2025,
    "reproduccionesMensuales": [20000]
  }
]
```

Conclusión: El desarrollo de este programa comprueba que la abstracción de datos puede extenderse hacia formatos de integración universal. Implementar un "parser" manual de JSON en C++ exige una comprensión lógica profunda sobre cómo identificar el inicio y fin de un objeto y cómo sanear cadenas de texto (eliminando comillas y delimitadores). El resultado es un código robusto que traslada estructuras dinámicas de la memoria RAM a un archivo estructurado sin necesidad de librerías externas.

GRAFO

Grafo con Algoritmo de Dijkstra y Exportación SVG

Introducción Esta práctica explora las estructuras de datos no lineales mediante la implementación de un Tipo de Dato Abstracto (TDA) "Grafo". A diferencia de listas o árboles, el grafo permite modelar redes complejas donde cualquier nodo puede conectarse con otro. El programa gestiona estas conexiones (aristas) asignándoles un peso (costo y tiempo), lo que permite no solo guardar y cargar la red en formato JSON, sino también calcular matemáticamente la ruta más corta usando el Algoritmo de Dijkstra y visualizar la topología exportándola a una imagen vectorial (SVG).

¿Cómo Funciona el Programa? El sistema integra múltiples conceptos algorítmicos y matemáticos, divididos en las siguientes áreas:

1.- El Modelo de Datos y Menú: Se utilizó un `std::map` para crear una Lista de Adyacencia dinámica, asociando el nombre de cada nodo (un string) con un `std::vector` de sus conexiones. El usuario interactúa mediante un menú a prueba de errores, blindado con `cin.ignore()` para evitar colapsos al mezclar entradas de texto y números.

2.- Implementación de Dijkstra y Retos Técnicos: Para calcular la ruta más corta basándose en el costo, se implementó una cola de prioridad (`priority_queue`). *Durante el desarrollo, se presentaron varios errores y complicaciones lógicas al implementar Dijkstra*, especialmente al intentar actualizar correctamente los costos acumulados y rastrear el historial de los "nodos padres" para reconstruir el camino final, lo cual se solucionó integrando mapas auxiliares.

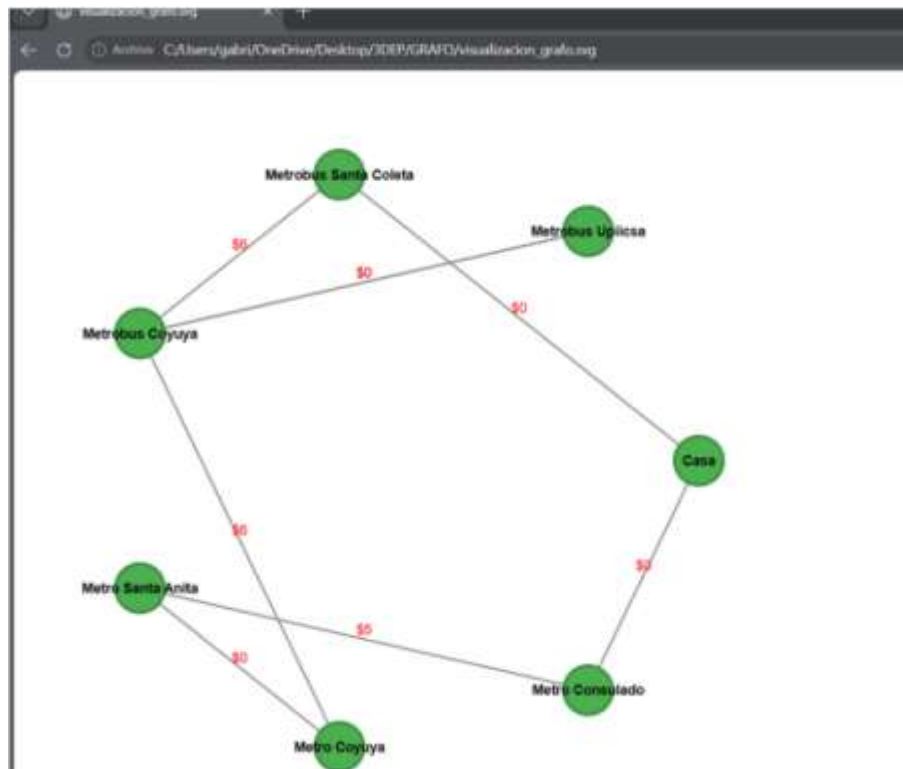

```

C:\Users\gabri\OneDrive\Desi x + v
RUTAS DEL GRAFO
Nodo: Casa
  -> Conecta con: Metro Consulado | Arista: A | Tiempo: 10 | Costo: $0
  -> Conecta con: Metrobus Santa Coleta | Arista: F | Tiempo: 5 | Costo: $0
Nodo: Metro Consulado
  -> Conecta con: Metro Santa Anita | Arista: B | Tiempo: 15 | Costo: $5
Nodo: Metro Coyuya
  -> Conecta con: Metrobus Coyuya | Arista: D | Tiempo: 2 | Costo: $6
Nodo: Metro Santa Anita
  -> Conecta con: Metro Coyuya | Arista: C | Tiempo: 3 | Costo: $0
Nodo: Metrobus Coyuya
  -> Conecta con: Metrobus Upiicsa | Arista: E | Tiempo: 10 | Costo: $0
  -> Conecta con: Metrobus Upiicsa | Arista: H | Tiempo: 10 | Costo: $0
Nodo: Metrobus Santa Coleta
  -> Conecta con: Metrobus Coyuya | Arista: G | Tiempo: 30 | Costo: $6
Nodo: Metrobus Upiicsa
SISTEMA DE RUTAS (TDA)
1. Agregar una conexion (Arista)
2. Mostrar todas las rutas
3. Calcular ruta mas corta (Dijkstra)
4. Guardar datos en JSON
5. Cargar datos desde JSON
6. Graficar
7. Salir
Elige una opcion: 3
Desde que nodo partes?: Casa
A que nodo quieres llegar?: Metrobus Upiicsa

RUTA MAS CORTA (DIJKSTRA)
Camino: Casa -> Metrobus Santa Coleta -> Metrobus Coyuya -> Metrobus Upiicsa
Costo total: $6

```

3.- Graficación Vectorial (SVG) y Coordenadas: El programa genera un archivo de imagen leyendo la estructura del grafo en memoria. Se *presentaron complicaciones significativas al momento de graficar debido al cálculo de las coordenadas (X, Y) en el lienzo*. Para resolver que los nodos no se dibujaran uno encima del otro, se tuvo que implementar trigonometría (cos y sin de la librería) para distribuir los vértices uniformemente en un radio circular perfecto.



4.- Persistencia (Lectura/Escritura JSON): Se programó un "parser" manual que lee el archivo rutas.json línea por línea. Usando la función limpiarValor, el programa recorta las llaves, elimina comas residuales y extrae la información pura para reconstruir el grafo en la memoria RAM tras cada reinicio.

```
{
  "rutas": [
    {
      "conexionNodo2Nodo": {
        "nodoInicial": "Casa",
        "nodoFinal": "Metro Consulado",
        "aristaConexion": "A",
        "tiempo": 10,
        "costo": 0
      }
    },
    {
      "conexionNodo2Nodo": {
        "nodoInicial": "Casa",
        "nodoFinal": "Metrobus Santa Coleta",
        "aristaConexion": "F",
        "tiempo": 5,
        "costo": 0
      }
    },
    {
      "conexionNodo2Nodo": {
        "nodoInicial": "Metro Consulado",
        "nodoFinal": "Metro Santa Anita",
        "aristaConexion": "B",
        "tiempo": 15,
        "costo": 5
      }
    },
    {
      "conexionNodo2Nodo": {
        "nodoInicial": "Metro Coyuya",
        "nodoFinal": "Metrobus Coyuya",
        "aristaConexion": "D",
        "tiempo": 2,
        "costo": 6
      }
    }
  ]
}
```

Conclusión: El desarrollo de este Grafo demuestra que la abstracción de datos va más allá de almacenar información; implica prepararla para ser procesada por algoritmos complejos. Superar los errores de rastreo en Dijkstra y resolver los conflictos de superposición de coordenadas matemáticas para la graficación SVG, demuestran un dominio sólido sobre la manipulación de TDAs, estructuras de control avanzadas y serialización de datos en C++.

DÍGRAFO

TDA Digrafo (Grafo Dirigido) y Serialización JSON

Introducción Esta práctica representa una evolución en el modelado de estructuras de datos no lineales al implementar un Digrafo (Grafo Dirigido). A diferencia de un grafo tradicional donde las conexiones son bidireccionales por defecto, un digrafo establece un "flujo" estricto; una conexión de un Nodo A hacia un Nodo B no implica que exista un camino de regreso. El objetivo es gestionar esta red jerárquica unidireccional, calcular trayectorias óptimas y lograr la persistencia de los datos estructurados en formato JSON.

¿Cómo Funciona el Programa? El código se organiza en módulos que manejan la lógica unidireccional, los cálculos matemáticos y la persistencia:

1.- Modelo Unidireccional: El TDA utiliza un `std::map` para asociar nodos con sus rutas. La diferencia lógica clave en el método `agregarConexion` es que el nodo destino se registra únicamente como receptor, creando verdaderas relaciones de "origen y destino (solo ida)".

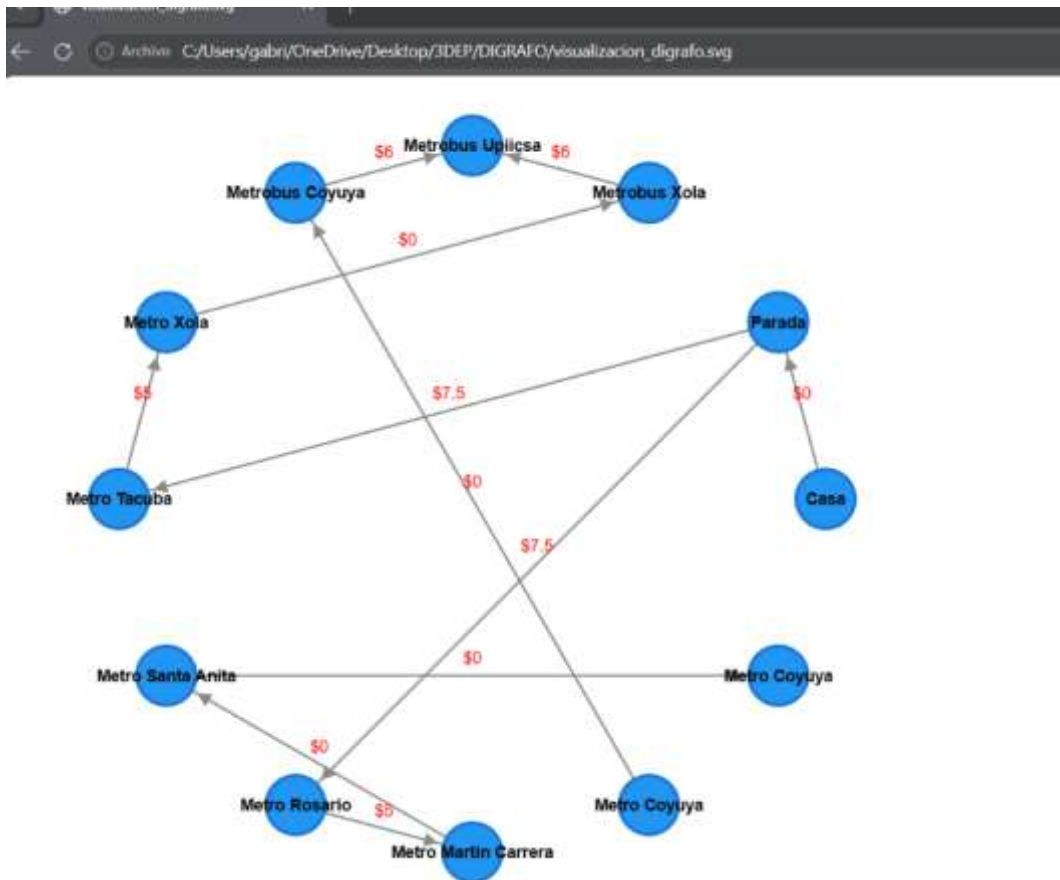
2.- Mejoras en el Algoritmo de Dijkstra: Se optimizó la implementación de Dijkstra respecto a versiones anteriores. En un digrafo, es común encontrar "callejones sin salida" (nodos a los que se puede llegar, pero de los que no se puede salir). El algoritmo mejorado maneja estos casos de forma robusta utilizando la cola de prioridad (`priority_queue`) y mapas de rastreo, deteniendo la búsqueda eficientemente y notificando si una ruta direccional es matemáticamente imposible de completar.

```
RUTAS DEL DIGRAFO
Nodo: Casa
-> Flujo hacia: Parada | Arista: A | Tiempo: 10 | Costo: $0
Nodo: Metro Coyuya
Nodo: Metro Coyuya
-> Flujo hacia: Metrobus Coyuya | Arista: F | Tiempo: 10 | Costo: $0
Nodo: Metro Martin Carrera
-> Flujo hacia: Metro Santa Anita | Arista: D | Tiempo: 25 | Costo: $0
Nodo: Metro Rosario
-> Flujo hacia: Metro Martin Carrera | Arista: C | Tiempo: 20 | Costo: $5
Nodo: Metro Santa Anita
-> Flujo hacia: Metro Coyuya | Arista: E | Tiempo: 7 | Costo: $0
Nodo: Metro Tacuba
-> Flujo hacia: Metro Xola | Arista: I | Tiempo: 20 | Costo: $5
Nodo: Metro Xola
-> Flujo hacia: Metrobus Xola | Arista: J | Tiempo: 3 | Costo: $0
Nodo: Metrobus Coyuya
-> Flujo hacia: Metrobus Upiicsa | Arista: G | Tiempo: 8 | Costo: $6
Nodo: Metrobus Upiicsa
Nodo: Metrobus Xola
-> Flujo hacia: Metrobus Upiicsa | Arista: K | Tiempo: 15 | Costo: $6
Nodo: Parada
-> Flujo hacia: Metro Rosario | Arista: B | Tiempo: 10 | Costo: $7.5
-> Flujo hacia: Metro Tacuba | Arista: H | Tiempo: 20 | Costo: $7.5
SISTEMA DE RUTAS (DIGRAFO)
1. Agregar una conexion dirigida (Flujo)
2. Mostrar todas las rutas
3. Calcular ruta mas corta (Dijkstra)
4. Guardar datos en JSON
5. Cargar datos desde JSON
6. Exportar Digrafo a imagen SVG
7. Salir
Elige una opcion: 3

ALGORITMO DE DIJKSTRA
Desde que nodo partes?: Casa
A que nodo quieres llegar?: Metrobus Upiicsa

RUTA MAS CORTA (DIJKSTRA)
Camino Dirigido: Casa -> Parada -> Metro Tacuba -> Metro Xola -> Metrobus Xola -> Metrobus Upiicsa
Costo total: $18.5
```

3.- Retos de Graficación en SVG: Durante la exportación visual volvieron a surgir complicaciones técnicas. Además de recalcular la distribución trigonométrica (cos y sin) para evitar que los nodos se empalmaran, el desafío principal fue representar la direccionalidad. Esto se solucionó incrustando código SVG avanzado (etiquetas y) para generar y apuntar flechas dinámicas (marker-end) al final de cada línea, haciendo evidente el flujo de los datos.



4.- Persistencia JSON: Se mantuvo la lectura y escritura manual del archivo .json iterando sobre el mapa de rutas y limpiando caracteres residuales con , asegurando que la topología de red pueda ser reconstruida tras cerrar el programa.

```
equipos.txt      discografia.json      rutas.json      rutas_digrafo.json
archivo  Editar  Ver

{
  "rutas": [
    {
      "conexionNodo2Nodo": {
        "nodoInicial": "Casa",
        "nodoFinal": "Parada",
        "aristaConexion": "A",
        "tiempo": 10,
        "costo": 0
      }
    },
    {
      "conexionNodo2Nodo": {
        "nodoInicial": "Metro Coyuya ",
        "nodoFinal": "Metrobus Coyuya",
        "aristaConexion": "F",
        "tiempo": 10,
        "costo": 0
      }
    },
    {
      "conexionNodo2Nodo": {
        "nodoInicial": "Metro Martin Carrera",
        "nodoFinal": "Metro Santa Anita",
        "aristaConexion": "D",
        "tiempo": 25,
        "costo": 0
      }
    },
    {
      "conexionNodo2Nodo": {
        "nodoInicial": "Metro Rosario",
        "nodoFinal": "Metro Martin Carrera",
        "aristaConexion": "C",
        "tiempo": 20,
        "costo": 5
      }
    }
  ]
}
```

Conclusión: El desarrollo de un Digrafo exige una comprensión más rigurosa sobre el flujo de los datos. Optimizar Dijkstra para que comprenda las restricciones direccionales y superar los obstáculos matemáticos y de renderizado vectorial para dibujar flechas en un lienzo SVG, demuestra una sólida capacidad para resolver problemas lógicos y manipular estructuras de datos avanzadas desde cero utilizando C++.

ÁRBOL

TDA Árbol y Jerarquías Visuales en SVG

Introducción Esta práctica final culmina el estudio de estructuras no lineales con la implementación de un Tipo de Dato Abstracto (TDA) Árbol. Un árbol es una especialización de un digrafo que obedece reglas lógicas estrictas: posee un nodo raíz principal y prohíbe explícitamente que un nodo hijo tenga más de un padre. El objetivo del programa es modelar estas jerarquías puras, calcular descendos

mediante el algoritmo de Dijkstra y persistir la información estructurada en formato JSON.

¿Cómo Funciona el Programa? El sistema integra validaciones lógicas y algoritmos de recorrido espacial divididos en los siguientes módulos:

1.- Restricción Jerárquica y TDA: A diferencia del grafo tradicional, el método de inserción valida activamente la regla del padre único utilizando un diccionario de control secundario (map padres). Si el usuario intenta conectar un nodo destino que ya cuenta con un ancestro registrado, el programa bloquea la acción para evitar ciclos y emite una alerta.

2.- Dijkstra Adaptado: Aunque la topología de un árbol garantiza que solo exista un único camino lógico entre la raíz y una hoja, se adaptó la cola de prioridad (priority_queue) para recorrer las ramas descendientemente. El programa evalúa los nodos permitidos y suma el costo de las aristas desde un ancestro hasta su descendiente.

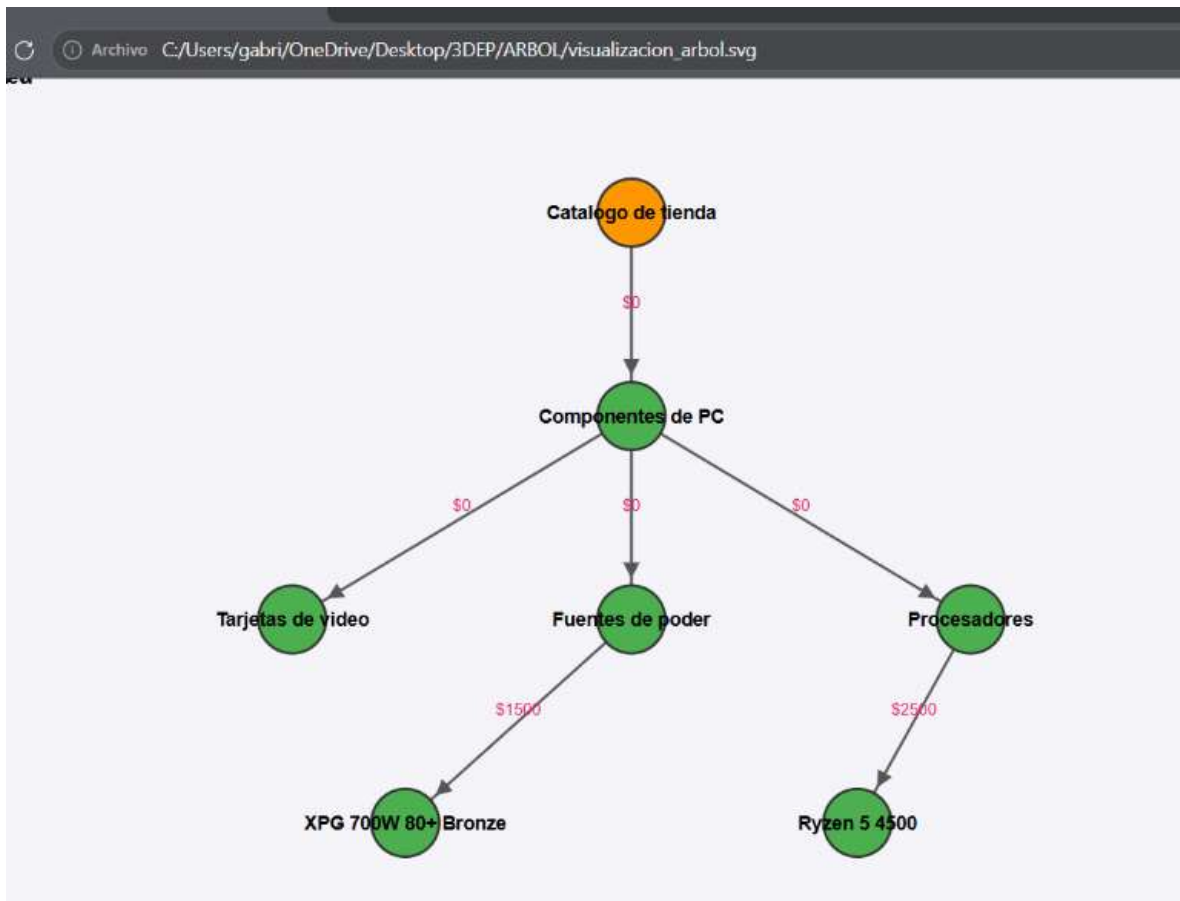
```
C:\Users\gabri\OneDrive\Desi x + v
ESTRUCTURA DEL ARBOL
[Raiz del Arbol]: Catalogo de tienda
Nodo Padre: Catalogo de tienda
  -> Hijo: Componentes de PC | Arista: Departamento | Tiempo: 1 | Costo: $0
Nodo Padre: Componentes de PC
  -> Hijo: Tarjetas de video | Arista: Subcategoria | Tiempo: 1 | Costo: $0
  -> Hijo: Fuentes de poder | Arista: Subcategoria | Tiempo: 1 | Costo: $0
  -> Hijo: Procesadores | Arista: Subcategoria | Tiempo: 1 | Costo: $0
Nodo Padre: Fuentes de poder
  -> Hijo: XPG 700W 80+ Bronze | Arista: Producto | Tiempo: 4 | Costo: $1500
Nodo Padre: Procesadores
  -> Hijo: Ryzen 5 4500 | Arista: Producto | Tiempo: 3 | Costo: $2500
Nodo Padre: Tarjetas de video
  -> Hijo: Asus RTX 3060 Ti | Arista: Producto | Tiempo: 2 | Costo: $5000
SISTEMA DE RUTAS (ARBOL TDA)
1. Agregar una conexion (Padre a Hijo)
2. Mostrar estructura jerarquica
3. Calcular ruta mas corta (Dijkstra)
4. Guardar datos en JSON
5. Cargar datos desde JSON
6. Exportar Arbol a imagen SVG
7. Salir
Elige una opcion: 3

--- BUSCAR RUTA (DIJKSTRA) ---
Desde que nodo (Padre/Ancastro) partes?: Componentes de PC
A que nodo (Hijo/Descendiente) quieres llegar?: Tarjetas de video

RUTA MAS CORTA (DIJKSTRA)
Rama: Componentes de PC -> Tarjetas de video
Costo total: $0
```

3.- Renderizado Jerárquico Vectorial (SVG): Para la exportación visual, se abandonó el enfoque trigonométrico circular. En su lugar, se programó un algoritmo de Búsqueda en Anchura (BFS) apoyado por una cola (queue) para recorrer la estructura. Esto permite identificar la raíz dinámica mediante la función

encontrarRaiz y asignar niveles (coordenadas "Y") a cada nodo, dibujando el organigrama de arriba hacia abajo y destacando la raíz en color naranja (#FF9800).



4.- Serialización JSON: El árbol asegura la retención de datos guardando la topología en el archivo rutas_arbol.json. Durante el proceso de lectura, el programa escanea las llaves y elimina las comillas residuales para reconstruir los arreglos en la memoria RAM de manera fiel.

Conclusión: La construcción de este Árbol comprueba cómo la adición de limitantes lógicas transforma el comportamiento de toda una red. Integrar un recorrido BFS para calcular coordenadas espaciales y generar un SVG jerárquico demuestra un dominio avanzado sobre la programación gráfica vectorial y el manejo de estructuras condicionales. El resultado es un sistema que blindo la integridad de los datos y serializa información de forma altamente visual y persistente.